Introducción a la programación con Python

Valladolid, 24 a 28 de febrero de 2025

Docente

- Francisco Javier Terceño García (Javi)
- Gerencia de Asistencia Sanitaria de Palencia - Atención Primaria.
- jtgarcia@saludcastillayleon.es

Organización del curso

- 5 días x 6 horas diarias
- L: 09:00 a 15:00
- MXJ: 08:30 a 14:30
- V: 08:00 a 14:00

Día 1: Introducción a Python

Bloque 1: ¿Qué es Python? Primeros pasos

¿Qué es un lenguaje de programación?

Un **lenguaje de programación** es "como un idioma" que utilizamos para comunicarnos con las computadoras. En vez de palabras, usamos un conjunto de instrucciones y reglas específicas que la máquina puede entender y ejecutar.

Imagina que una computadora es un robot gigante que solo entiende órdenes muy precisas. Si quieres que haga algo, tienes que darle instrucciones paso a paso, y esas instrucciones las escribes en un lenguaje de programación.

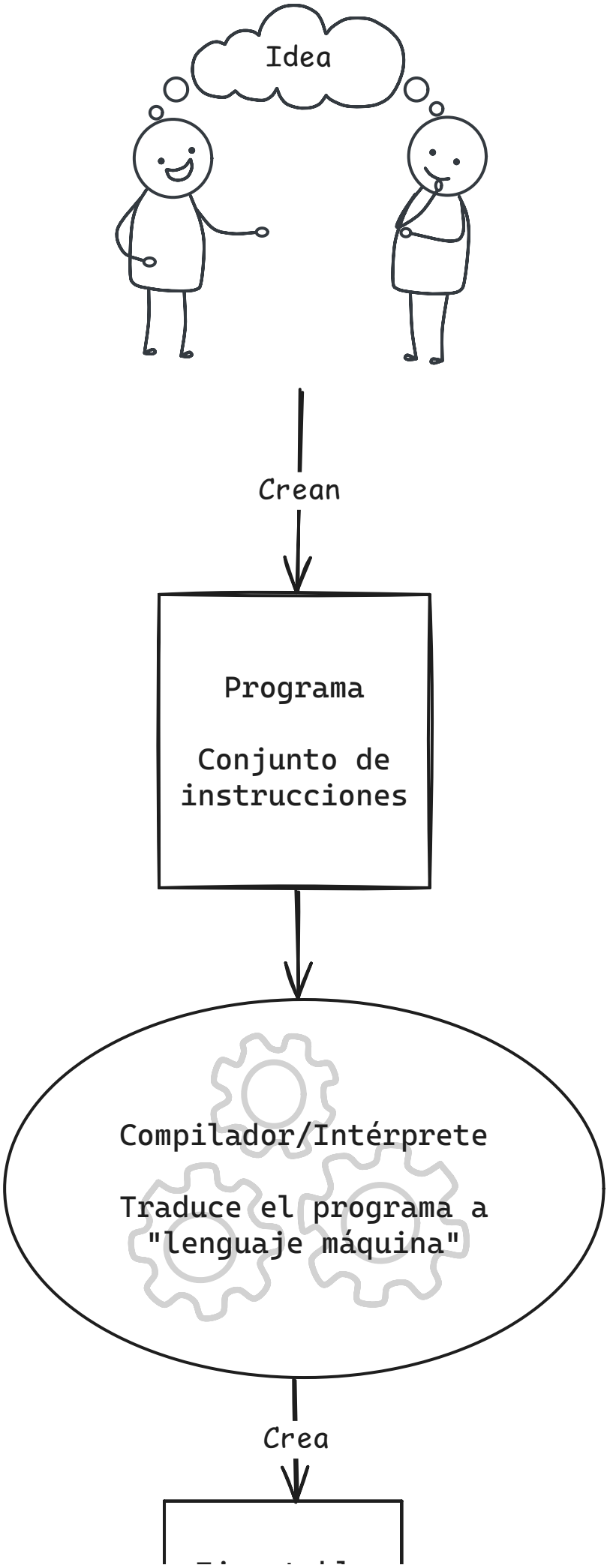
¿Para qué sirven los lenguajes de programación?

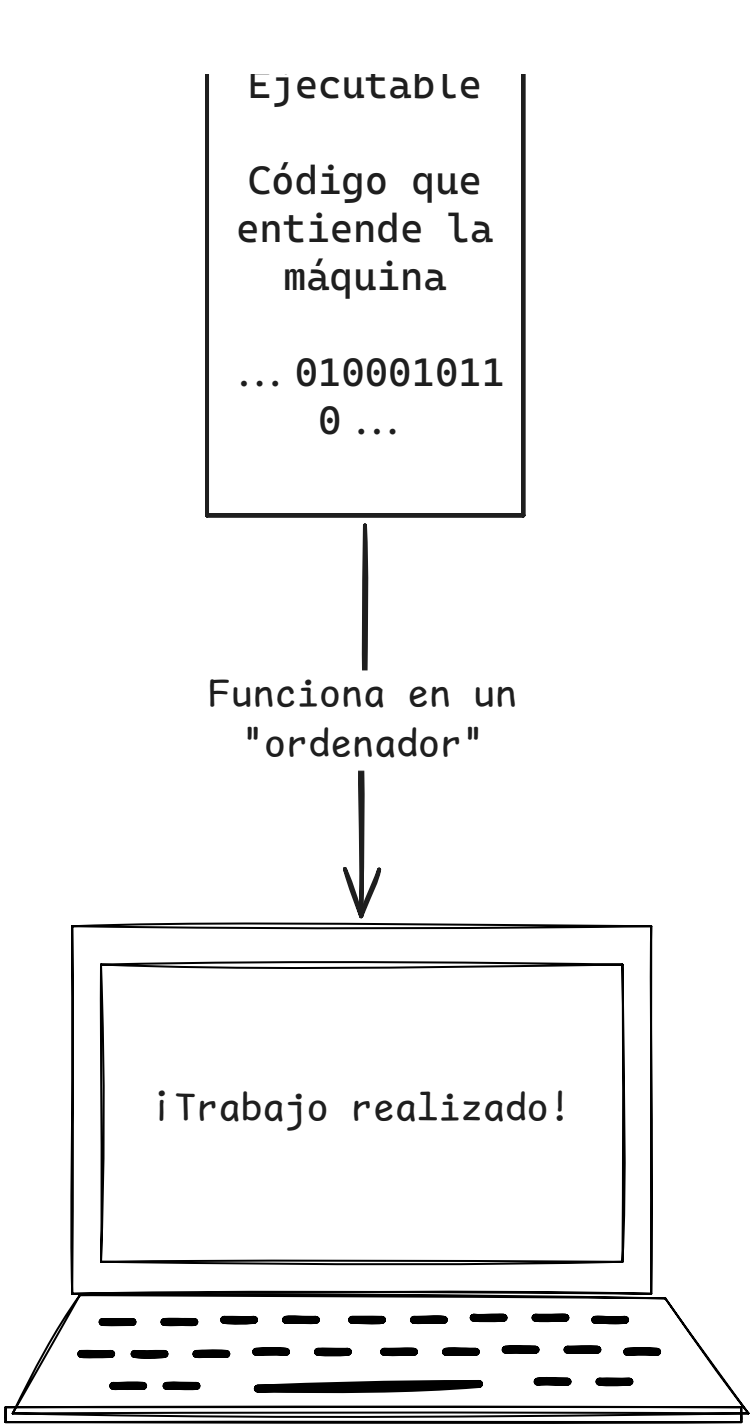
- **Crear software:** Desde simples aplicaciones hasta sistemas operativos complejos.
- **Desarrollar sitios web:** Tanto la parte visual como la lógica detrás de ellos.
- **Controlar robots y dispositivos:** Automatizar tareas y crear sistemas inteligentes.
- **Analizar datos:** Obtener información valiosa a partir de grandes conjuntos de datos.
- **Y mucho más...**

¿Cómo funcionan los lenguajes de programación?

Cuando escribes código en un lenguaje de programación, estás creando un programa. Este programa es traducido a un lenguaje que la computadora puede entender directamente, conocido como código máquina, a través de un compilador o un intérprete.

programa-lenguaje-maquina


Idea
Crean
Programa
Conjunto de instrucciones
Compilador/Intérprete
Traduce el programa a "lenguaje máquina"
Crea



Excalidraw Data

Text Elements

Programa

Conjunto de instrucciones

Compilador/Intérprete

Traduce el programa a "lenguaje máquina"

Ejecutable

Código que entiende la máquina

...0100010110...

Crean

Idea

Funciona en un "ordenador"

Crea

¡Trabajo realizado!

¿Por qué Python? Ventajas y aplicaciones?

¿Por qué Python?

Python destaca por su **sintaxis clara y legible**, lo que lo hace más fácil de aprender y entender en comparación con otros lenguajes. Esta característica, junto con su **gran comunidad y ecosistema de librerías**, lo convierten en una herramienta poderosa y versátil.

Ventajas de Python

- **Fácil de aprender:** Su sintaxis es similar al lenguaje natural, lo que facilita la curva de aprendizaje, incluso para aquellos que se inician en la programación.
- **Versatilidad:** Se utiliza en una amplia gama de aplicaciones, desde desarrollo web y análisis de datos hasta inteligencia artificial y aprendizaje automático.
- **Amplia biblioteca estándar:** Incluye un gran número de módulos que proporcionan funcionalidades listas para usar, lo que agiliza el desarrollo.
- **Gran comunidad:** Cuenta con una comunidad activa y en constante crecimiento, lo que significa que hay muchos recursos, tutoriales y soporte disponibles.
- **Software libre:** Es gratuito y de código abierto, lo que permite su libre uso y modificación.

- **Portabilidad:** El código escrito en Python puede ejecutarse en diferentes sistemas operativos sin mayores modificaciones.

Aplicaciones de Python

Las posibilidades con Python son prácticamente ilimitadas. Algunas de sus aplicaciones más comunes incluyen:

- **Desarrollo web:** Frameworks como Django y Flask permiten crear aplicaciones web robustas y escalables.
- **Ciencia de datos y análisis:** Librerías como NumPy, Pandas y Matplotlib son fundamentales para el análisis de datos y la visualización.
- **Aprendizaje automático e inteligencia artificial:** Frameworks como TensorFlow y PyTorch se utilizan para desarrollar modelos de machine learning y deep learning.
- **Automatización de tareas:** Python es excelente para automatizar tareas repetitivas, como procesamiento de archivos, extracción de datos de sitios web o envío de correos electrónicos.
- **Desarrollo de juegos:** Con librerías como Pygame, se pueden crear juegos 2D y 3D.

En resumen

Python es un lenguaje de programación versátil, fácil de aprender y con una gran comunidad. Sus ventajas lo hacen ideal para una amplia gama de aplicaciones, desde proyectos personales hasta desarrollos empresariales a gran escala.

Historia de Python

Los comienzos

La historia de Python comienza a finales de los años 80, cuando un programador holandés, llamado Guido van Rossum, trabajando en el Centro de Investigación Matemática y Computación de los Países Bajos

(CWI), decidió crear un nuevo lenguaje de programación como proyecto personal durante las vacaciones de Navidad.

- **Influencias:** Van Rossum se inspiró en otros lenguajes como [ABC](https://es.wikipedia.org/wiki/ABC_(lenguaje_de_programaci%C3%B3n), que buscaba ser fácil de aprender y usar. Sin embargo, ABC tenía algunas limitaciones que Guido quería superar.
- **Nacimiento de Python:** En diciembre de 1989, comenzó a trabajar en lo que sería Python. El nombre "Python" fue elegido en honor al grupo de comedia británica Monty Python, del cual Van Rossum era un gran fan.
- **Primera versión:** La primera versión pública de Python, la 0.9.0, fue lanzada en febrero de 1991.

Crecimiento y evolución

- **Características clave:** Desde el principio, Python destacó por su sintaxis clara y legible, inspirada en el lenguaje natural. Esta característica, junto con su énfasis en la legibilidad del código, lo hizo muy atractivo para los programadores.
- **Adopción gradual:** Python comenzó a ganar popularidad entre los desarrolladores, especialmente en la comunidad de software libre. Su uso se extendió más allá de los círculos académicos y comenzó a ser utilizado en proyectos comerciales.
- **Versiones importantes:** A lo largo de los años, se lanzaron varias versiones importantes de Python, cada una con nuevas características y mejoras. La versión 2.0, lanzada en el año 2000, introdujo importantes cambios como el manejo de excepciones y la programación orientada a objetos.
- **Python 3:** En 2008, se lanzó Python 3, una versión con cambios significativos respecto a las versiones anteriores. Aunque Python 3 introdujo muchas mejoras, su adopción fue más lenta debido a la necesidad de actualizar el código existente.

Situación actual

En la actualidad, Python es uno de los lenguajes de programación más populares del mundo. Se utiliza en empresas de todos los tamaños, desde startups hasta grandes corporaciones. Su influencia se ha extendido a campos como la ciencia de datos, el aprendizaje automático, la inteligencia artificial, el desarrollo web y muchos otros.

Guido van Rossum, el creador de Python, sigue siendo una figura influyente en la comunidad de Python, aunque se retiró oficialmente del desarrollo del lenguaje en 2018.

Resumen

La historia de Python es una historia de éxito basada en un diseño sencillo, una comunidad apasionada y una constante evolución. Su capacidad para adaptarse a las necesidades cambiantes de los desarrolladores lo ha convertido en un lenguaje de programación imprescindible en el mundo moderno.

Versiones

Guido van Rossum ideó el lenguaje Python a finales de los 80 y comenzó a implementarlo en diciembre de 1989. En febrero de 1991 publicó la primera versión pública, la versión 0.9.0. La versión 1.0 se publicó en enero de 1994, la versión 2.0 se publicó en octubre de 2000 y la versión 3.0 se publicó en diciembre de 2008.

Hasta 2018, el desarrollo de Python estaba dirigido personalmente por Guido van Rossum y bajo el paraguas de la fundación Python Software Foundation. En julio de 2018 Guido van Rossum anunció que dejaría de dirigir el desarrollo de Python. Desde 2019 el desarrollo de Python está dirigido por un consejo de dirección de cinco miembros elegidos entre los desarrolladores de Python y que se renueva anualmente.

En octubre de 2019 se decidió cambiar el ritmo de publicación de nuevas versiones de Python de los anteriores 18 meses a 12 meses. Desde Python 3.8 (publicado en octubre de 2019) se publica un nueva versión de Python cada mes de octubre.

Las versiones de Python se identifican por tres números X.Y.Z, en la que:

- X corresponde a las grandes versiones de Python (1, 2 y 3), incompatibles entre sí: Los principales cambios introducidos en Python 2 fueron las cadenas Unicode, las comprensiones de listas, las asignaciones aumentadas, los nuevos métodos de cadenas y el recolector de basura para referencias cíclicas. Los principales cambios introducidos en Python 3 fueron la separación entre cadenas Unicode y datos binarios, la función print(), cambios en la sintaxis, tipos de datos, comparadores, etc. Por el momento, no hay planes de crear una nueva versión Python 4, incompatible con las anteriores.
- Y corresponde a versiones importantes en las que se introducen novedades en el lenguaje pero manteniendo la compatibilidad (salvo excepciones). Desde la versión 2.0 hasta 2019, las versiones X.Y se publicaron aproximadamente cada año y medio y se mantenían durante cinco años, excepto la versión 2.7, que se mantuvo durante diez años, hasta el 1 de enero de 2020 (aunque se publicó una versión final en abril de 2020). En octubre de 2019 se decidió cambiar el ritmo de publicación de nuevas versiones de Python de los anteriores 18 meses a 12 meses, manteniendo los 5 años de mantenimiento. Desde Python 3.8 (publicado en octubre de 2019) se publica un nueva versión de Python cada mes de octubre.
- Z corresponde a versiones menores que se publican durante el período de mantenimiento, en las que sólo se corrigen errores durante el primer año y fallos de seguridad en los cuatro restantes. La publicación de las versiones .Z es más irregular, porque el descubrimiento de fallos de seguridad puede provocar la necesidad de publicación de una nueva versión en cualquier momento, pero hay un calendario previsto de versiones .Z. En principio, en el primer año después de la publicación de la versión inicial X.Y.0, se publican versiones .Z cada dos meses, tanto en forma de código fuente como de instaladores. Tras la publicación de la siguiente versión X.(Y+1).0 se publica una versión X.Y.Z proporcionando instaladores, pero las versiones .Z que se publican hasta completar los cinco años de mantenimiento ya sólo se publican en forma de código fuente. Además, normalmente se publica una última versión X.Y.Z justo antes de que una versión X.Y deje de mantenerse. Al margen de las versiones publicadas por la PSF, algunas empresas comerciales

ofrecen el mantenimiento de versiones antiguas una vez acabado el mantenimiento oficial.

Es posible tener instalados en el ordenador varias versiones de Python pero, salvo que sea necesario para la ejecución de programas o paquetes incompatibles, se recomienda instalar siempre la última versión disponible.

Instalación de Python

- Python 2 vs Python 3
- Existen múltiples versiones de Python
  - ¿Qué versión instalo?

MS-Windows

- Descargar y ejecutar instalador
  - No marcar la instalación para todos los usuarios
  - No marcar la adición al `PATH`
  - Elegir modo personalizado
    - Marcar únicamente `pip` y `Tcl/Tk`
    - Elegir el destino oportuno y no marcar nada en las opciones avanzadas
- Añadir al `PATH`
  - el directorio elegido para la instalación
  - y su subdirectorio scripts
- Comprobar la instalación
  - Abrir una consola en el directorio de instalación y ejecutar

    ```
    $ python --version
    ```

  - Windows 10 lanza la tienda de aplicaciones en lugar del ejecutable de Python
    - Configuración / Alias de ejecución de aplicaciones
    - Desactivar Python

Configurar el entorno para tener múltiples versiones de Python

- Crear un directorio `bin`
- Añadir el directorio `bin` al `PATH`
- Crear allí un `bat` para cada versión de Python instalada

¿Qué se ha instalado? - Biblioteca estándar vs bibliotecas de terceros >

- Instalador de módulos/paquetes/bibliotecas
  - PIP >

Proxy

El intérprete de Python es una aplicación de consola y necesita conocer los datos del proxy, para lo que usa las variables de entorno

```
# JCYL
HTTP_PROXY=http://proxy.jcyl.es:80/
HTTPS_PROXY=http://proxy.jcyl.es:80/

# SACYL
HTTP_PROXY=http://usuario:password@proxy.sacyl.es:3128/
HTTPS_PROXY=http://usuario:password@proxy.sacyl.es:3128/
```

Instalación de Microsoft Visual Studio Code

Se trata de un editor de código, pero que mediante el uso de extensiones puede transformarse en un `IDE`.
Es de código abierto y puede descargarse libremente desde la web del fabricante.

1. Descargaremos la versión `zip` disponible para nuestra plataforma.
2. La descomprimiremos en la ubicación que nos interese.
3. Añadiremos dicha ubicación al `PATH` del sistema.

Podemos consultar la extensa documentación del editor en su pagina web.

Estructura básica

La estructura básica de un programa en Python es bastante sencilla y legible, lo que contribuye a su facilidad de aprendizaje. Los elementos fundamentales que componen un programa en Python son los siguientes:

Elementos básicos de un programa en Python

Comentarios

- Se utilizan para explicar el código y hacerlo más comprensible.
- Se inician con el símbolo `#`.
- Todo el texto después del `#` en esa línea se considera un comentario y no es ejecutado.

Ejemplo:

```
# Este es un comentario de una línea
print("Hola, mundo!")  # Este es otro comentario
```

Variables

- Se utilizan para almacenar datos.
- No es necesario declarar el tipo de dato de una variable antes de usarla.
- Se asignan valores utilizando el signo igual.

Ejemplo:

```
nombre = "Juan"
edad = 30
```

Tipos de datos

- Python tiene varios tipos de datos integrados, como números enteros (`int`), números de punto flotante (`float`), cadenas de texto (`str`), booleanos (`bool`), listas, tuplas, diccionarios y conjuntos.

Ejemplo:

```
numero_entero = 10
numero_decimal = 3.14
cadena = "Hola"
booleano = True
```

Operadores

- Se utilizan para realizar operaciones matemáticas, lógicas y de comparación.

Ejemplos:

```
suma = 2 + 3
resta = 5 - 2
multiplicacion = 4 * 2
division = 10 / 2
```

Instrucciones

- Son las acciones que el programa realiza.
- Se escriben una por línea.

Ejemplos:

```
print("Hola, mundo!")
nombre = input("¿Cuál es tu nombre? ")
```

Bloques de código

- Se utilizan para agrupar varias instrucciones.
- Se definen por la sangría (espacios al inicio de una línea).

Ejemplos:

```
if edad >= 18:
     # mayor de edad
     print("Eres mayor de edad.")
else:
     # no es mayor de edad
     print("Eres menor de edad.")
```

Estructura general de un programa Python

Un programa Python típico puede tener la siguiente estructura:

```
# Comentario al inicio del programa

# Definición de funciones (si las hay)

# Cuerpo principal del programa
if __name__ == "__main__":
    # Instrucciones del programa principal
    # ...
```

Ejemplo completo

```
# Programa que calcula el área de un círculo
import math # Importa el módulo math para usar la constante pi

radio = float(input("Ingrese el radio del círculo: "))
area = math.pi * radio * radio

print("El área del círculo es:", area)
```

Resumen

La estructura de un programa Python es clara y fácil de seguir.
La sangría juega un papel fundamental en la definición de bloques de código.
Al entender estos conceptos básicos, podrás comenzar a escribir tus propios programas en Python.

- **Práctica:**
  - Experimentar con el intérprete interactivo de Python.
  - Crear un archivo Python y ejecutar un simple "¡Hola, Mundo!".

Bloque 2: Sintaxis básica

¿Qué es una vairable?

En programación, una variable es como un contenedor o una etiqueta que le das a un espacio en la memoria de la computadora para almacenar un valor. Este valor puede ser de diferentes tipos, como números, texto, listas, etc.

Las variables son fundamentales en programación porque permiten

- **Almacenar datos:** Las variables guardan los datos con los que trabaja un programa.
- **Manipular datos:** Puedes cambiar el valor de una variable durante la ejecución del programa, lo que permite realizar cálculos y operaciones.
- **Reutilizar datos:** Una vez que has almacenado un valor en una variable, puedes usar esa variable en diferentes partes del programa sin tener que volver a escribir el valor.
- **Hacer el código más legible:** Usar nombres descriptivos para las variables hace que el código sea más fácil de entender.

En Python, puedes crear una variable simplemente asignándole un valor. No es necesario declarar el tipo de variable, Python lo infiere automáticamente. La asignación se realiza mediante el símbolo igual "="

Tipos de datos básicos.

- Enteros: `int`

```
edad = 30
```

- Decimales: `float`

```
pi = 3.14
```

- Cadenas de caracteres: `str`

```
# pueden ir entre comillas simples o dobles
nombre = 'Javi'
apellidos = "López García"
```

```
# la triple comilla, simple o doble, para multilínea
direccion = '''Avda. de la Libertad
             Madrid
             España'''
```

- Booleanos: `bool`

```
# hay dos posibles valores: True y False
activo = True
inactivo = False
```

Operadores aritméticos, de comparación y lógicos.

- Aritméticos
  - Suma: `+`
  - Resta: `-`
  - Multiplicación: `*`
  - División: `/`
  - División entera: `//`
  - Potencias: `**`
  - Módulo: `%`
- Comparativos
  - Igualdad: ==
  - Distinción: `!=`
  - Mayor, mayor o igual: `>`, `>=`
  - Menor, menor o igual: `<`, `<=`
- Logicos
  - Y: `and`
  - O: `or`
  - No: `not`
- Identidad
  - Es, no es: `is`, `is not`
- Membresía:
  - Está en: `in`
  - No está en: `not in`
- A nivel bit o binarios

    - O: `|`
    - Y: `&`
    - No: `~`
    - O exclusivo: `^`
    - Desplazamientos: `<<`, `>>`
- Walrus, o morsa
    - Asignación "en línea": `:=`

Comentarios en Python.

- Los comentarios se marcan con el símbolo `#`
- **Práctica:**
    - Realizar operaciones matemáticas básicas.
    - Crear variables de diferentes tipos y realizar operaciones entre ellas.
    - Utilizar operadores de comparación para tomar decisiones simples.

Bloque 3: Entrada y salida de datos

La función `input`

La función `input()` en Python es una herramienta fundamental para interactuar con el usuario. Su propósito principal es permitir que un programa reciba datos introducidos por el usuario a través del teclado.

¿Cómo funciona?

1. **Solicita entrada:** Cuando se ejecuta la función `input()`, el programa se detiene y espera a que el usuario escriba algo en la consola.
2. **Mensaje opcional:** Puedes incluir un mensaje dentro de los paréntesis de `input()` para indicar al usuario qué tipo de información se espera. Este mensaje se mostrará en la consola antes de que el usuario introduzca su respuesta.
3. **Captura la entrada:** Una vez que el usuario presiona la tecla "Enter", la función `input()` captura todo lo que el usuario escribió y lo devuelve como una cadena de texto (string).
4. **Almacena la entrada:** Normalmente, la entrada del usuario se almacena en una variable para su uso posterior en el programa.

Sintaxis básica:

```
variable = input("Mensaje opcional: ")
```

Ejemplos:

- **Solicitar el nombre del usuario:**

```
nombre = input("Introduce tu nombre: ")
print("Hola, " + nombre + "!")
```

- **Solicitar la edad del usuario:**

```
edad = input("Introduce tu edad: ")
#Nota importante: input siempre devuelve una cadena de texto.
#Si se quiere trabajar con numeros, hay que convertirla.
edad = int(edad)
print("Tienes " + str(edad) + " años.")
```

Puntos importantes:

- La función `input()` siempre devuelve una cadena de texto, incluso si el usuario introduce números. Si necesitas trabajar con números, debes convertir la entrada utilizando las funciones `int()` (para números enteros) o `float()` (para números decimales).
- Es importante manejar posibles errores de conversión. Por ejemplo, si intentas convertir una entrada no numérica a un entero, se producirá un error. Puedes utilizar bloques `try` y `except` para manejar estas situaciones.
- Cuando se trabaja con datos numéricos, es muy importante realizar la conversión del tipo de dato, ya que si no se hace, se producirán errores al intentar realizar operaciones aritméticas.

**La función `print`

La función `print()` en Python es una herramienta esencial para mostrar información en la consola o en la salida estándar. Se utiliza para visualizar

resultados, mensajes, valores de variables y cualquier otro tipo de información que necesites mostrar durante la ejecución de un programa.

¿Cómo funciona?

1. **Muestra información:** La función `print()` toma uno o más argumentos y los muestra en la consola.
2. **Argumentos opcionales:** Puedes pasar diferentes tipos de datos como argumentos a `print()`, incluyendo cadenas de texto, números, variables y expresiones.
3. **Formato de salida:** `print()` formatea automáticamente los argumentos para mostrarlos de forma legible.
4. **Salto de línea:** Por defecto, `print()` añade un salto de línea al final de la salida, lo que significa que la siguiente llamada a `print()` mostrará la información en una nueva línea.

Sintaxis básica:

```
print(objeto(s), sep=separador, end=final, file=archivo,
flush=booleano)
```

Argumentos:

- **objeto(s):** Los objetos que se van a mostrar. Pueden ser uno o varios, separados por comas.
- **sep:** Un separador opcional que se utiliza para separar los objetos mostrados. Por defecto, es un espacio en blanco.
- **end:** Un carácter opcional que se añade al final de la salida. Por defecto, es un salto de línea (`\n`).
- **file:** Un objeto archivo opcional donde se escribirá la salida. Por defecto, es la salida estándar (`sys.stdout`).
- **flush:** Un valor booleano opcional que indica si se debe forzar el vaciado del búfer de salida.

Ejemplos:

- **Mostrar una cadena de texto:**

```
print("Hola, mundo!")
```

- **Mostrar el valor de una variable:**

```
nombre = "Juan"
print("Mi nombre es", nombre)
```

- **Mostrar varios objetos separados por comas:**

```
edad = 30
print("Tengo", edad, "años.")
```

- **Mostrar objetos sin salto de línea:**

```
print("Hola", end=" ")
print("mundo!")
```

- **Mostrar objetos con un separador personalizado:**

```
print("Manzana", "Naranja", "Plátano", sep=", ")
```

- **Usando f-strings (cadenas formateadas):**

```
nombre = "Ana"
edad = 25
print(f"Mi nombre es {nombre} y tengo {edad} años.")
```

Puntos importantes:

- La función `print()` es muy útil para depurar programas, ya que te permite inspeccionar el valor de las variables en diferentes puntos de la ejecución.
- Las cadenas formateadas (f-strings) son una forma conveniente de insertar variables y expresiones dentro de cadenas de texto.
- Puedes redirigir la salida de `print()` a un archivo en lugar de la consola utilizando el argumento `file`.

¿Qué son las f-strings?

Las f-strings son cadenas de texto que tienen una "f" al principio, seguidas de comillas (simples, dobles o triples). Dentro de la cadena, puedes insertar variables o expresiones encerrándolas entre llaves `{}`. Python evaluará automáticamente estas expresiones y las reemplazará con sus valores correspondientes.

Sintaxis básica:

```
f"cadena de texto {variable o expresión}"
```

Ejemplos:

- **Insertar variables:**

```
nombre = "Ana"
edad = 30
print(f"Mi nombre es {nombre} y tengo {edad} años.")
```

- **Insertar expresiones:**

```
num1 = 10
num2 = 5
print(f"La suma de {num1} y {num2} es {num1 + num2}.")
```

- **Formatear números:**

```
pi = 3.14159
print(f"El valor de pi con dos decimales es {pi:.2f}.")
```

Ventajas de las f-strings:

- **Legibilidad:** Las f-strings son más fáciles de leer y escribir que otros métodos de formateo de cadenas, como `%` o `.format()`.
- **Concisión:** Permiten insertar variables y expresiones directamente dentro de las cadenas, lo que reduce la cantidad de código necesario.

- **Velocidad:** Las f-strings son más rápidas que otros métodos de formateo de cadenas.
- **Flexibilidad:** Admiten una amplia gama de expresiones y formatos.

Características adicionales:

- **Depuración:** Desde Python 3.8, las f-strings admiten el especificador "=" para mostrar tanto el nombre de la variable como su valor:

```
x = 10
print(f"{x=}")  # Salida: x=10
```

- **Llamadas a funciones:** Puedes llamar a funciones directamente dentro de las f-strings:

```
def cuadrado(num):
    return num**2

print(f"El cuadrado de 5 es {cuadrado(5)}.")
```

- **Práctica:**
  - Crear un programa que solicite al usuario su nombre y edad, y luego imprima un saludo personalizado.
  - Convertir un número de segundos a horas, minutos y segundos.

Bloque 4: estructuras de control condicionales

Las estructuras de control son esenciales para crear programas útiles y complejos. Sin ellas, los programas serían lineales y limitados en su capacidad de resolver problemas. Gracias a las estructuras de control, los programas pueden tomar decisiones, repetir acciones y adaptarse a diferentes situaciones, lo que los hace más potentes y versátiles.

Sentencia `if`

La sentencia `if` es una estructura de control condicional que te permite ejecutar un bloque de código solo si se cumple una determinada condición. Es como una pregunta que le haces a tu programa: "Si esta condición es verdadera, entonces haz esto".

Sintaxis de la sentencia `if`

La sintaxis básica de la sentencia `if` es la siguiente:

```
if condición:
    # Código a ejecutar si la condición es verdadera
```

- `condición`: Es una expresión que se evalúa como verdadera (`True`) o falsa (`False`).
- `# Código a ejecutar si la condición es verdadera`: Es el bloque de código que se ejecuta solo si la condición es verdadera. Este bloque debe estar sangrado, generalmente con 4 espacios, para indicar que forma parte de la sentencia `if` .

Ejemplo de la sentencia if:

```
edad = 20

if edad >= 18:
    print("Eres mayor de edad.")
```

En este ejemplo, la condición `edad >= 18` se evalúa como verdadera porque la variable `edad` tiene un valor de 20. Por lo tanto, el código dentro del bloque `if` se ejecuta y se imprime el mensaje "Eres mayor de edad." en la pantalla.

¿Cómo funciona la sentencia `if` ?

1. Se evalúa la condición.
2. Si la condición es verdadera (`True`), se ejecuta el código dentro del bloque `if` .
3. Si la condición es falsa (`False`), el código dentro del bloque `if` se omite y el programa continúa con la siguiente instrucción después del bloque `if` .

Sentencias `if` anidadas

Puedes colocar una sentencia `if` dentro de otra sentencia `if` para crear condiciones más complejas. Esto se conoce como sentencias `if` anidadas.

```
edad = 20
tiene_carnet = True

if edad >= 18:
    if tiene_carnet:
        print("Puedes conducir.")
    else:
        print("No puedes conducir porque no tienes carnet.")
else:
    print("Eres menor de edad.")
```

Sintaxis completa

```
if condicion_1:
    # si se cumple condicion_1
     # se ejecuta este bloque de código
elif condicion_2:
    # si no se cumple condicion_1
     # y se cumple condición_2
     # se ejecuta este bloque de código
...
# puede haber varios bloques elif
else:
     # si no se ha cumplido ninguna de la condiciones
     # se ejecuta este bloque de código
```

Manejo de excepciones

Una excepción es un evento que ocurre durante la ejecución de un programa que interrumpe el flujo normal de las instrucciones. Las excepciones son una forma de manejar errores y situaciones inesperadas que pueden surgir durante la ejecución del código. Estos eventos pueden ser desde errores de sintaxis hasta problemas más complejos como la falta de memoria o la conexión a una base de datos.

¿Por qué es importante el manejo de excepciones?

El manejo de excepciones te permite crear programas que puedan anticipar y responder a estos eventos inesperados. En lugar de que el programa falle y

se cierre abruptamente, puedes "atrapar" la excepción y ejecutar un código específico para manejar la situación.

¿Cómo se manejan las excepciones en Python?

Python utiliza bloques `try-except` para manejar excepciones. El bloque `try` contiene el código que puede generar una excepción, y el bloque `except` contiene el código que se ejecuta si se produce una excepción.

```
try:
    # Código que puede generar una excepción
    resultado = 10 / 0  # ¡¡¡Error!!! división por cero
except ZeroDivisionError:
    # Código para manejar la excepción ZeroDivisionError
    print("¡Error! No se puede dividir por cero.")
```

En este ejemplo, el bloque `try` contiene una operación de división que puede generar una excepción `ZeroDivisionError` si el denominador es cero. Si ocurre esta excepción, el código dentro del bloque `except` se ejecuta y se imprime un mensaje de error.

Tipos de excepciones

Python tiene varios tipos de excepciones incorporadas, como `ZeroDivisionError`, `TypeError`, `ValueError`, `IndexError`, etc. Cada tipo de excepción representa un error específico que puede ocurrir durante la ejecución del programa.

Bloques `try-except` múltiples

Puedes usar múltiples bloques `except` para manejar diferentes tipos de excepciones:

```
try:
    # Código que puede generar diferentes excepciones
    valor = int("Hola")  # Error de tipo
    lista = [1, 2, 3]
    print(lista[5])  # Error de índice
except TypeError:
    print("¡Error! El valor debe ser un número entero.")
```

```
except IndexError:
    print("¡Error! El índice está fuera de rango.")
```

Bloque `finally`

El bloque `finally` se ejecuta siempre, independientemente de si se produce una excepción o no. Se utiliza para realizar tareas de limpieza, como cerrar archivos o liberar recursos.

```
try:
    # Código que puede generar una excepción
    archivo = open("archivo.txt", "r")
    # ... puede haber uno o varios bloques except
finally:
    # Código que se ejecuta siempre
    archivo.close()
```

En resumen

El manejo de excepciones es una técnica fundamental en programación que te permite crear programas robustos y confiables. Al utilizar bloques `try-except`, puedes anticipar y responder a eventos inesperados, evitando que tus programas fallen y se cierren abruptamente.

- **Práctica:**
  - Crear un programa que determine si un número es positivo, negativo o cero.
  - Implementar un programa que verifique si un año es bisiesto.
  - Escribir un programa que solicite al usuario un número y lo divida por otro número ingresado. Manejar la excepción `ZeroDivisionError`.
  - Leer un archivo que puede no existir y manejar la excepción `FileNotFoundError`.

Bloque 5: estructuras de control repetitivas (parte 1)

La sentencia `for`

La sentencia `for` es una estructura de control iterativa que te permite repetir un bloque de código un número determinado de veces o para cada elemento de una secuencia, como una lista, tupla o cadena.

Sintaxis de la sentencia `for`

```
for elemento in secuencia:
    # Código a ejecutar para cada elemento
```

- `elemento`: Es una variable que toma el valor de cada elemento de la secuencia en cada iteración.
- `secuencia`: Es una lista, tupla, cadena u otro objeto iterable que contiene los elementos sobre los que se va a iterar.
- `# Código a ejecutar para cada elemento`: Es el bloque de código que se ejecuta para cada elemento de la secuencia. Este bloque debe estar sangrado, generalmente con 4 espacios, para indicar que forma parte de la sentencia `for`.

Ejemplo de la sentencia `for`

```
frutas = ["manzana", "plátano", "cereza"]

for fruta in frutas:
    print(fruta)
```

En este ejemplo, la variable `fruta` toma el valor de cada elemento de la lista `frutas` en cada iteración. El código dentro del bloque `for` se ejecuta para cada fruta, imprimiendo su nombre en la pantalla.

¿Cómo funciona la sentencia `for`?

```
for elemento in secuencia:
    # bloque con instrucciones
    # que se ejecutan para cada elemento de la secuencia
```

1. Se itera sobre cada elemento de la secuencia.
2. En cada iteración, la variable `elemento` toma el valor del elemento actual de la secuencia.

3. Se ejecuta el código dentro del bloque `for`.
4. Se repiten los pasos 2 y 3 hasta que se hayan recorrido todos los elementos de la secuencia.

Uso de la función `range()`

La función `range()` es muy útil para generar secuencias de números que se pueden usar en bucles `for`.

```
for i in range(5):
    print(i)
```

Este código imprimirá los números del 0 al 4 en la pantalla.
La expresión `range(numero)` devuelve una secuencia de números que comienza en 0 y termina en número-1

Sentencias `for` anidadas

Puedes colocar una sentencia `for` dentro de otra sentencia `for` para crear bucles anidados. Esto es útil cuando necesitas iterar sobre múltiples secuencias.

```
for i in range(3):
    for j in range(2):
        print(i, j)
```

Este código imprimirá todas las combinaciones posibles de los números 0, 1 y 2 con los números 0 y 1.

En resumen

La sentencia `for` es una herramienta esencial en programación que te permite repetir un bloque de código un número determinado de veces o para cada elemento de una secuencia. Su capacidad para iterar sobre colecciones de datos hace que tus programas sean más eficientes y capaces de realizar tareas repetitivas de manera automática.

- **Práctica:**
    - Imprimir los números del 1 al 10.

- Calcular la suma de los números del 1 al 100.

Bloque 6: estructuras de control repetitivas (parte 2)

La sentencia `while`

La sentencia `while` es una estructura de control iterativa que te permite repetir un bloque de código mientras se cumpla una determinada condición. Es como un bucle que se ejecuta una y otra vez hasta que la condición deja de ser verdadera.

Sintaxis de la sentencia `while`

La sintaxis básica de la sentencia `while` es la siguiente:

```
while condición:
    # Código a ejecutar mientras la condición sea verdadera
```

- `condición`: Es una expresión que se evalúa como verdadera (`True`) o falsa (`False`).
- `# Código a ejecutar mientras la condición sea verdadera`: Es el bloque de código que se ejecuta repetidamente mientras la condición sea verdadera. Este bloque debe estar sangrado, generalmente con 4 espacios, para indicar que forma parte de la sentencia `while` .

Ejemplo de la sentencia `while`

```
contador = 0

while contador < 5:
    print(contador)
    contador += 1
```

En este ejemplo, la variable `contador` se inicializa en 0.
La condición `contador < 5` se evalúa antes de cada iteración.
Mientras la condición sea verdadera, el código dentro del bloque `while` se ejecuta, imprimiendo el valor de `contador` y luego incrementándolo en 1.
El bucle se detiene cuando `contador` llega a 5, ya que la condición `contador < 5` se vuelve falsa.

¿Cómo funciona la sentencia `while`?

1. Se evalúa la condición.
2. Si la condición es verdadera (`True`), se ejecuta el código dentro del bloque `while` .
3. Se vuelve al paso 1 para evaluar la condición de nuevo.
4. Si la condición es falsa (`False`), el bucle `while` termina y el programa continúa con la siguiente instrucción después del bloque `while` .

Bucles infinitos

Es importante tener cuidado con los bucles infinitos, que son bucles `while` cuya condición nunca se vuelve falsa. Esto puede ocurrir si olvidas actualizar una variable que se utiliza en la condición o si la condición siempre se evalúa como verdadera.

```
# Ejemplo de bucle infinito (¡cuidado!)
while True:
    print("Este bucle nunca terminará.")
```

Para detener un bucle infinito, puedes presionar Ctrl+C en la terminal o cerrar la ventana del programa.

Uso de `break` y `continue`

Dentro de un bucle `while` , o `for` , puedes utilizar las sentencias `break` y `continue` para controlar el flujo del bucle:

- `break` : Sale del bucle inmediatamente.
- `continue` : Salta a la siguiente iteración del bucle, omitiendo el resto del código dentro del bloque para la iteración actual.

Ejemplo de uso de `break` y `continue`

```
contador = 0

while contador < 10:
    contador += 1
    if contador == 5:
        continue  # Salta a la siguiente iteración si contador es 5
```

```
    print(contador)
    if contador == 8:
        break  # Sale del bucle si contador es 8
```

En resumen

La sentencia `while` es una herramienta esencial en programación que te permite repetir un bloque de código mientras se cumpla una determinada condición. Su capacidad para crear bucles que se ejecutan hasta que una condición cambia hace que tus programas sean más dinámicos y capaces de realizar tareas repetitivas de manera automática.

- **Teoría:**
  - El bucle `while`: ejecutando un bloque de código mientras una condición sea verdadera
  - Controlando un bucle
    - La instrucción `break`: forzando la salida de un bucle
    - La sentencia `continue`: cancelando la iteración actual
- **Práctica:**
  - Crear un programa que pida al usuario números hasta que ingrese 0.
  - Implementar un programa que adivine un número aleatorio.

Día 2: Otros tipos y estructuras de datos básicos

Ya hemos visto los tipos de datos básicos, pero además, Python ofrece otros tipos más complejos

- Listas
- Tuplas
- Diccionarios

Bloque 1: listas

Imagina una lista como un contenedor versátil donde puedes guardar una colección ordenada de elementos. Cada elemento en la lista tiene una posición, conocida como índice, que nos permite acceder a él de manera individual.

Características clave de las listas:

- **Ordenadas:** Los elementos en una lista mantienen el orden en el que se agregan.
- **Mutables:** Puedes modificar las listas después de su creación, agregando, eliminando o cambiando elementos.
- **Heterogéneas:** Las listas pueden contener elementos de diferentes tipos de datos (números, cadenas de texto, booleanos, incluso otras listas).

¿Cómo crear una lista?

Puedes crear una lista utilizando corchetes `[]` y separando los elementos por comas, o con la función `list`:

```
mi_lista = [10, "Hola", 3.14, True]
```

Acceder a los elementos de una lista

Puedes acceder a los elementos de una lista utilizando su índice. El índice comienza desde 0 para el primer elemento:

```
print(mi_lista[0]) # Imprime 10
print(mi_lista[1]) # Imprime "Hola"
```

Modificar una lista

Las listas son mutables, lo que significa que puedes cambiar sus elementos:

- **Agregar elementos:**

```
mi_lista.append(20)  # Agrega 20 al final de la lista
mi_lista.insert(1, "Mundo")  # Inserta "Mundo" en la posición 1
```

- **Eliminar elementos:**

```
del mi_lista[0] # Elimina el primer elemento
mi_lista.remove("Mundo") # Elimina la primera ocurrencia de "Mundo"
mi_lista.pop() # Elimina y devuelve el último elemento
```

- **Modificar elementos:**

```
mi_lista = [10, "Hola", 3.14, True]
mi_lista[0] = 20   # cambio el primer elemento
print(mi_lista)    # [20, "Hola", 3.14, True]
```

¿Para qué se utilizan las listas?

Las listas son increíblemente útiles para una variedad de tareas:

- **Almacenar colecciones de datos:** Puedes guardar información relacionada en una lista, como nombres de estudiantes, precios de productos, etc.
- **Procesar datos:** Puedes iterar sobre una lista para realizar operaciones en cada elemento.
- **Crear estructuras de datos más complejas:** Las listas pueden contener otras listas, lo que permite crear estructuras de datos más sofisticadas.

Ejemplo práctico

```
nombres = ["Ana", "Juan", "María"]
edades = [25, 30, 28]
for i in range(len(nombres)):
     print(nombres[i], "tiene", edades[i], "años.")
```

En este ejemplo, se utilizan dos listas para almacenar nombres y edades, y luego se itera sobre ellas para imprimir información personalizada.

En resumen

Las listas son una herramienta fundamental en Python que te permite trabajar con colecciones de datos de manera eficiente y flexible. Su capacidad para almacenar diferentes tipos de datos y su mutabilidad las convierten en una estructura de datos esencial para cualquier programador de Python.

- **Teoría:**
  - Creación de listas.
  - Acceso a elementos (indexación, slicing).
  - Modificación de listas (`append` , `insert` , `remove` , `pop`).

    - Operaciones con listas (`concatenación` , `repetición`).
    - Métodos de listas (`sort` , `reverse` , `count` , `index`).
- **Práctica:**
    - Crear una lista de nombres y ordenarla alfabéticamente.
    - Crear una lista de números y encontrar el número más grande.
    - Implementar una función para invertir una lista.

Bloque 2: tuplas

Una tupla es una colección ordenada e inmutable de elementos. Similar a una lista, una tupla puede contener elementos de diferentes tipos de datos (números, cadenas de texto, booleanos, etc.). Sin embargo, la principal diferencia es que una vez creada, una tupla no se puede modificar.

Características clave de las tuplas:

- **Ordenadas:** Los elementos en una tupla mantienen el orden en el que se definen.
- **Inmutables:** Una vez creada, no puedes cambiar los elementos de una tupla, ni agregar nuevos, ni eliminar existentes.
- **Heterogéneas:** Pueden contener elementos de diferentes tipos de datos.

¿Cómo crear una tupla?

Puedes crear una tupla utilizando paréntesis `()` y separando los elementos por comas, o con la función `tuple` :

```
mi_tupla = (10, "Hola", 3.14, True)
```

Acceder a los elementos de una tupla

Puedes acceder a los elementos de una tupla utilizando su índice, al igual que con las listas:

```
print(mi_tupla[0])  # Imprime 10
print(mi_tupla[1])  # Imprime "Hola"
```

¿Por qué usar tuplas?

Aunque las tuplas no se pueden modificar, su inmutabilidad las hace útiles en ciertas situaciones:

- **Datos protegidos:** Si necesitas asegurarte de que los datos no se modifiquen accidentalmente, puedes almacenarlos en una tupla.
- **Mayor eficiencia:** En algunos casos, las tuplas pueden ser más eficientes en términos de memoria y rendimiento que las listas.
- **Devolver múltiples valores en funciones:** Las funciones pueden devolver múltiples valores empaquetándolos en una tupla.

Ejemplo práctico

```
coordenadas = (10, 20) # coordenada en 2 dimensiones
print(coordenadas[0])  # Imprime 10
print(coordenadas[1])  # Imprime 20
```

Diferencias entre listas y tuplas

Característica	Lista	Tupla
Mutabilidad	Mutable (se puede modificar)	Inmutable (no se puede modificar)
Sintaxis	Corchetes `[]`	Paréntesis `()`
Uso	Almacenar colecciones de datos variables	Almacenar datos protegidos o constantes

En resumen

Las tuplas son una herramienta útil en Python para trabajar con colecciones de datos que no deben ser modificadas. Su inmutabilidad las hace ideales para proteger datos y garantizar la integridad de la información.

- **Teoría:**
  - Creación de tuplas.
  - Acceso a elementos.
  - Inmutabilidad de las tuplas.
  - Usos comunes de las tuplas.
- **Práctica:**

- Crear una tupla con los días de la semana.
- Empaquetar y desempaquetar valores en tuplas.
- Utilizar tuplas como claves en diccionarios.

Bloque 3: diccionarios

Un diccionario en Python es una estructura de datos que te permite almacenar pares de **clave-valor**. Imagina un diccionario real: buscas una palabra (clave) y encuentras su definición (valor). Los diccionarios en Python funcionan de manera similar.

Características clave de los diccionarios:

- **Clave-valor:** Cada elemento en un diccionario está compuesto por una clave única y su valor asociado.
- **No ordenados:** A diferencia de las listas, los diccionarios no mantienen un orden específico de los elementos.
- **Mutables:** Puedes modificar los diccionarios después de su creación, agregando, eliminando o cambiando elementos.

¿Cómo crear un diccionario?

Puedes crear un diccionario utilizando llaves `{}` y separando los pares clave-valor por dos puntos `:` , o la función `dict`

```
mi_diccionario = {"nombre": "Juan", "edad": 30, "ciudad": "Madrid"}
```

Acceder a los valores de un diccionario

Puedes acceder a los valores de un diccionario utilizando su clave:

```
print(mi_diccionario["nombre"])  # Imprime "Juan"
print(mi_diccionario["edad"])  # Imprime 30
```

Modificar un diccionario

Los diccionarios son mutables, lo que significa que puedes cambiar sus elementos:

- **Agregar elementos:**

```
mi_diccionario["profesion"] = "Programador"
```

- **Eliminar elementos:**

```
del mi_diccionario["ciudad"]
```

- **Cambiar elementos:**

```
mi_diccionario["edad"] = 31
```

¿Para qué se utilizan los diccionarios?

Los diccionarios son muy útiles para una variedad de tareas:

- **Almacenar información estructurada:** Puedes guardar datos relacionados en un diccionario, como información de contacto, perfiles de usuario, etc.
- **Representar objetos:** Los diccionarios pueden representar objetos con sus atributos y valores.
- **Conteo y frecuencia:** Puedes usar diccionarios para contar la frecuencia de elementos en una lista u otro iterable.

Ejemplo práctico

```
persona = {"nombre": "Ana", "edad": 25, "intereses": ["leer",
"viajar"]}

print(persona["nombre"])  # Imprime "Ana"
print(persona["intereses"])  # Imprime ["leer", "viajar"]

for interes in persona["intereses"]:
    print(interes)
```

- **Teoría:**
  - Creación de diccionarios.
  - Acceso a valores (claves).
  - Modificación de diccionarios.

    - Métodos de diccionarios (keys, values, items).
- **Práctica:**
    - Crear un diccionario para almacenar la información de contacto de varias personas.
    - Iterar sobre un diccionario y mostrar todas las claves y valores.
    - Contar la frecuencia de cada palabra en un texto utilizando un diccionario.

Bloque 4: conjuntos

Un conjunto en Python es una colección desordenada de elementos únicos. Esto significa que un conjunto no puede contener elementos duplicados y no mantiene un orden específico de los elementos. Los conjuntos son similares a los conjuntos matemáticos y se utilizan para realizar operaciones como unión, intersección y diferencia.

Características clave de los conjuntos:

- **Elementos únicos:** Un conjunto no puede contener elementos duplicados. Si intentas agregar un elemento que ya existe en el conjunto, no se agregará.
- **Desordenados:** Los conjuntos no mantienen un orden específico de los elementos. No puedes acceder a los elementos de un conjunto utilizando un índice.
- **Mutables:** Puedes modificar los conjuntos después de su creación, agregando o eliminando elementos.

¿Cómo crear un conjunto?

Puedes crear un conjunto utilizando llaves `{}` o la función `set()`:

```
mi_conjunto = {1, 2, 3, 4, 5}  # Utilizando llaves
otro_conjunto = set([1, 2, 3])  # Utilizando la función set()
```

Operaciones con conjuntos

Los conjuntos ofrecen varias operaciones útiles:

- **Unión:** Combina dos conjuntos en uno nuevo que contiene todos los elementos de ambos conjuntos.

```
conjunto1 = {1, 2, 3}
conjunto2 = {3, 4, 5}
union = conjunto1 | conjunto2  # Utilizando el operador |
# o
union = conjunto1.union(conjunto2)  # Utilizando el método union()
print(union)  # Imprime {1, 2, 3, 4, 5}
```

- **Intersección:** Encuentra los elementos comunes a dos conjuntos.

```
interseccion = conjunto1 & conjunto2  # Utilizando el operador &
# o
interseccion = conjunto1.intersection(conjunto2)  # Utilizando el
método intersection()
print(interseccion)  # Imprime {3}
```

- **Diferencia:** Encuentra los elementos que están en un conjunto pero no en otro.

```
diferencia = conjunto1 - conjunto2  # Utilizando el operador -
# o
diferencia = conjunto1.difference(conjunto2)  # Utilizando el método
difference()
print(diferencia)  # Imprime {1, 2}
```

¿Para qué se utilizan los conjuntos?

Los conjuntos son útiles para una variedad de tareas:

- **Eliminar duplicados:** Puedes convertir una lista en un conjunto para eliminar elementos duplicados.
- **Verificar pertenencia:** Puedes verificar si un elemento pertenece a un conjunto de manera eficiente.
- **Operaciones matemáticas:** Puedes realizar operaciones de conjuntos como unión, intersección y diferencia.

Ejemplo práctico

```
numeros = [1, 2, 2, 3, 4, 4, 5]
conjunto_numeros = set(numeros)  # Elimina duplicados
print(conjunto_numeros)  # Imprime {1, 2, 3, 4, 5}
```

- **Teoría:**
  - Creación de conjuntos.
  - Operaciones con conjuntos (unión, intersección, diferencia).
  - Métodos de conjuntos (add, remove, clear).
- **Práctica:**
  - Crear dos conjuntos de números y encontrar los elementos comunes y únicos.
  - Eliminar elementos duplicados de una lista utilizando conjuntos.

Bloque 5: definición por compresión

Imagina que necesitas crear una lista con los cuadrados de los números pares del 1 al 10. Podrías hacerlo con un bucle `for` y una condición `if`, pero la definición por comprensión te permite lograrlo en una sola línea de código.

La definición por comprensión es una expresión que te permite crear una nueva colección iterando sobre una colección existente y aplicando una condición o transformación a cada elemento.

Sintaxis de la definición por comprensión

La sintaxis general de la definición por comprensión es la siguiente:

```
[expresión for elemento in iterable if condición]
```

- `expresión`: Es la operación o transformación que se aplica a cada elemento.
- `elemento`: Es la variable que representa cada elemento de la colección iterable.
- `iterable`: Es la colección sobre la que se itera (lista, tupla, conjunto, etc.).

- `condición`: Es una condición opcional que se debe cumplir para que un elemento se incluya en la nueva colección.

Ejemplos de definición por comprensión

- **Lista de cuadrados de números pares:**

```
cuadrados_pares = [x**2 for x in range(1, 11) if x % 2 == 0]
print(cuadrados_pares)  # Imprime [4, 16, 36, 64, 100]
```

- Conjunto de vocales en una cadena:

```
cadena = "Hola mundo"
vocales = {x for x in cadena if x in "aeiou"}
print(vocales)  # Imprime {'a', 'o', 'u'}
```

- Diccionario de nombres y longitudes:

```
nombres = ["Ana", "Juan", "María"]
longitudes = {nombre: len(nombre) for nombre in nombres}
print(longitudes)  # Imprime {'Ana': 3, 'Juan': 4, 'María': 5}
```

Ventajas de la definición por comprensión

- **Concisión:** Reduce la cantidad de código necesario para crear colecciones.
- **Legibilidad:** Facilita la comprensión del código al expresar la lógica de creación de colecciones de forma clara y directa.
- **Eficiencia:** En algunos casos, la definición por comprensión puede ser más eficiente que los bucles `for` tradicionales.

En resumen

La definición por comprensión es una herramienta poderosa en Python que te permite crear colecciones de manera concisa y elegante. Su capacidad para expresar la lógica de creación de colecciones en una sola línea de código hace que tu código sea más legible y eficiente.

- **Teoría:**

    - Sintaxis de la compresión de listas.
    - Creación de listas a partir de otras iterables.
- **Práctica:**
    - Crear una lista con los cuadrados de los números del 1 al 10 usando comprensión de listas.
    - Filtrar una lista de números para obtener solo los números pares.

Bloque 6: funciones

¿Qué son las funciones?

Imagina que tienes un robot que realiza tareas específicas. Le das una orden (llamada "argumento"), el robot realiza la tarea (ejecuta el "código de la función") y te da un resultado (el "valor de retorno"). ¡Eso es una función!

En Python, una función es un bloque de código reutilizable que realiza una tarea específica. Las funciones te permiten:

- **Organizar tu código:** Dividir un programa complejo en partes más pequeñas y manejables.
- **Reutilizar código:** Evitar escribir el mismo código una y otra vez.
- **Abstraer detalles:** Ocultar la complejidad de una tarea detrás de un nombre sencillo.

Sintaxis de una función

```
def nombre_de_la_funcion(parametro1, parametro2, ...):
    """Docstring: Descripción de la función"""
    # Código de la función
    return valor_de_retorno
```

- `def` : Palabra clave para definir una función.
- `nombre_de_la_funcion` : Nombre que le das a la función (debe ser descriptivo).
- `parametro1` , `parametro2` , ...: Valores que la función recibe como entrada (opcional).
- `"""Docstring: Descripción de la función"""` : Comentario que describe la función (¡muy recomendable!).

- `# Código de la función`: Bloque de código que realiza la tarea.
- `return valor_de_retorno`: Valor que la función devuelve como resultado (opcional).

Ejemplo práctico: ¡Calculadora de áreas!

```
def calcula_area_rectangulo(base, altura):
    """Calcula el área de un rectángulo."""
    area = base * altura
    return area

base = 10
altura = 5
area = calcula_area_rectangulo(base, altura)
print(f"El área del rectángulo es: {area}")  # Imprime 50
```

¡Manos a la obra! Ejercicios prácticos

1. **Función saludo personalizado:** Crea una función que reciba un nombre como argumento y devuelva un saludo personalizado (por ejemplo, "¡Hola, Juan!").
2. **Función número mayor:** Crea una función que reciba dos números como argumentos y devuelva el número mayor.
3. **Función tabla de multiplicar:** Crea una función que reciba un número como argumento e imprima la tabla de multiplicar de ese número del 1 al 10.

¡Profundizando en las funciones!

- **Argumentos opcionales:** Puedes definir parámetros con valores por defecto para que sean opcionales al llamar a la función.

```
def saludar(nombre="Usuario"):
    print(f"¡Hola, {nombre}!")

saludar()  # Imprime "¡Hola, Usuario!"
saludar("Ana")  # Imprime "¡Hola, Ana!"
```

- **Argumentos con nombre:** Puedes especificar el nombre de los parámetros al llamar a la función para evitar confusiones.

```
calcular_area_rectangulo(altura=5, base=10)
```

- **Funciones dentro de funciones:** Puedes definir funciones dentro de otras funciones para organizar mejor tu código.

```
def funcion_externa():
    def funcion_interna():
        print("¡Hola desde la función interna!")
    funcion_interna()
    print("¡Hola desde la función externa!")

funcion_externa()
```

¡El poder de la recursión!

Una función puede llamarse a sí misma. ¡Esto se llama recursión! Es útil para resolver problemas que se pueden dividir en problemas más pequeños del mismo tipo.

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5))  # Imprime 120
```

Argumentos múltiples

En Python, puedes definir funciones que reciban un número variable de argumentos. Esto se logra utilizando los símbolos `*` y `**`.

`*args`

El símbolo `*` se utiliza para pasar un número variable de argumentos posicionales a una función. Estos argumentos se almacenan en una tupla llamada `args`.

```python
def sumar_numeros(*args):
    """Suma un número variable de números."""
    suma = 0
    for numero in args:
        suma += numero
    return suma

resultado = sumar_numeros(1, 2, 3, 4, 5)
print(resultado)  # Imprime 15
```

`kwargs`**

El símbolo `**` se utiliza para pasar un número variable de argumentos con nombre (o palabras clave) a una función. Estos argumentos se almacenan en un diccionario llamado `kwargs`.

```python
def imprimir_info(**kwargs):
    """Imprime información de una persona."""
    for clave, valor in kwargs.items():
        print(f"{clave}: {valor}")

imprimir_info(nombre="Juan", edad=30, ciudad="Madrid")
# Imprime:
# nombre: Juan
# edad: 30
# ciudad: Madrid
```

Combinación de argumentos

Puedes combinar argumentos posicionales, argumentos con nombre, `*args` y `**kwargs` en una misma función.

```python
def funcion_mixta(a, b, *args, nombre="Usuario", **kwargs):
    """Función con diferentes tipos de argumentos."""
    print(f"a: {a}")
    print(f"b: {b}")
    print(f"args: {args}")
    print(f"nombre: {nombre}")
    print(f"kwargs: {kwargs}")
```

```
funcion_mixta(1, 2, 3, 4, nombre="Ana", edad=25, ciudad="Barcelona")
# Imprime:
# a: 1
# b: 2
# args: (3, 4)
# nombre: Ana
# kwargs: {'edad': 25, 'ciudad': 'Barcelona'}
```

Consejos para usar argumentos múltiples y con nombre

- **Legibilidad:** Utiliza argumentos con nombre para que tu código sea más fácil de entender.
- **Flexibilidad:** Utiliza `*args` y `**kwargs` para crear funciones que puedan recibir un número variable de argumentos.
- **Documentación:** Documenta claramente el uso de cada argumento en la docstring de la función.

¡Consejos para dominar las funciones!

- **Nombres descriptivos:** Elige nombres claros y descriptivos para tus funciones.
- **Docstrings:** Escribe docstrings para explicar qué hace cada función.
- **Modularidad:** Divide tu código en funciones pequeñas y reutilizables.
- **Practica:** ¡La mejor forma de aprender es practicando!

Día 3: Programación orientada a objetos y módulos

Bloque 1: introducción a la programación orientada a objetos (POO)

¿Qué es un objeto en Python?

En Python, un objeto es una entidad que combina datos (atributos) y acciones (métodos) que operan sobre esos datos. Puedes imaginar un objeto como una "cosa" que tiene propiedades y puede realizar tareas.

Características clave de los objetos:

- **Atributos:** Son los datos que describen al objeto. Por ejemplo, un objeto "coche" podría tener atributos como "color", "marca" y "velocidad".
- **Métodos:** Son las funciones que definen las acciones que el objeto puede realizar. Por ejemplo, un objeto "coche" podría tener métodos como "acelerar", "frenar" y "girar".

¿Por qué usar objetos?

Los objetos son útiles porque permiten:

- **Organizar el código:** Los objetos agrupan datos y acciones relacionadas, lo que hace que el código sea más fácil de entender y mantener.
- **Reutilizar código:** Puedes crear objetos que sirvan como "plantillas" para crear otros objetos similares, lo que ahorra tiempo y esfuerzo.
- **Modelar el mundo real:** Los objetos te permiten representar entidades del mundo real en tu código, lo que facilita la resolución de problemas complejos.

Ejemplo de objeto en Python

```
class Coche:  # Definimos la clase "Coche"
    def __init__(self, color, marca, velocidad):  # Constructor del objeto
        self.color = color  # Atributos del objeto
        self.marca = marca
        self.velocidad = velocidad

    def acelerar(self):  # Métodos del objeto
        self.velocidad += 10

    def frenar(self):
        self.velocidad -= 10

mi_coche = Coche("rojo", "Toyota", 0)  # Creamos un objeto "mi_coche"

print(mi_coche.color)  # Accedemos a los atributos del objeto
print(mi_coche.velocidad)
```

```
mi_coche.acelerar()  # Llamamos a los métodos del objeto
print(mi_coche.velocidad)
```

En este ejemplo, la clase `Coche` define la estructura y el comportamiento de los objetos "coche". El constructor `__init__` inicializa los atributos del objeto, y los métodos `acelerar` y `frenar` definen las acciones que el objeto puede realizar.

¿Por qué en Python todo es un objeto?

La respuesta corta es: porque Python es un lenguaje de programación orientado a objetos (POO) puro. Esto significa que **todo** en Python, desde los tipos de datos más básicos hasta las funciones y los módulos, se representa como objetos.

¿Qué significa esto?

- **Datos y comportamiento:** Cada objeto en Python combina datos (atributos) y acciones (métodos) que operan sobre esos datos.
- **Instancias de clases:** Los objetos son instancias de clases, que son como "plantillas" que definen la estructura y el comportamiento de los objetos.
- **Herencia y polimorfismo:** La POO permite que los objetos hereden atributos y métodos de otras clases (herencia) y que diferentes objetos respondan a los mismos métodos de maneras diferentes (polimorfismo).

¿Qué ventajas ofrece esto?

- **Consistencia:** Al ser todo un objeto, Python ofrece una forma consistente de trabajar con diferentes tipos de datos y entidades.
- **Reusabilidad:** Los objetos se pueden reutilizar en diferentes partes del programa, lo que ahorra tiempo y esfuerzo.
- **Modularidad:** La POO fomenta la creación de código modular y organizado, lo que facilita el mantenimiento y la colaboración en proyectos grandes.
- **Abstracción:** Los objetos permiten abstraer conceptos complejos en entidades más simples y manejables.

Ejemplos de objetos en Python

- **Números:** Incluso los números enteros y decimales son objetos en Python.

```
x = 10
print(x.real)  # Accedemos al atributo "real" del objeto número
```

- **Listas, tuplas, diccionarios:** Estas estructuras de datos son objetos que contienen otros objetos.
- **Funciones:** ¡Incluso las funciones son objetos en Python!

```
def saludar(nombre):
    print("Hola,", nombre)

print(type(saludar))  # Imprime <class 'function'>
```

- **Teoría:**
  - Concepto de clase y objeto.
  - Atributos y métodos.
  - Instanciación de objetos.
- **Práctica:**
  - Crear una clase `Perro` con atributos como nombre, raza y edad.
  - Crear objetos de la clase `Perro` y modificar sus atributos.

Bloque 2: métodos especiales

Método `__init__`

¿Qué es el método `__init__`?

El método `__init__` es un método especial en Python que se ejecuta automáticamente cuando se crea un nuevo objeto (instancia) de una clase. Su principal función es inicializar los atributos (datos) del objeto.

¿Por qué es importante el método `__init__`?

El método `__init__` es importante porque permite:

- **Inicializar objetos:** Define los valores iniciales de los atributos de un objeto al momento de su creación.

- **Recibir argumentos:** Puede recibir argumentos que se utilizan para configurar el objeto durante su inicialización.
- **Crear objetos consistentes:** Asegura que todos los objetos de una clase tengan los atributos necesarios y estén en un estado válido desde el principio.

Sintaxis del método `__init__`

La sintaxis del método `__init__` es la siguiente:

```
def __init__(self, argumento1, argumento2, ...):
    self.atributo1 = argumento1
    self.atributo2 = argumento2
    # ...
```

- `self`: Es una referencia al objeto que se está creando. Se utiliza para acceder a los atributos del objeto dentro del método `__init__`.
- `argumento1`, `argumento2`, etc.: Son los argumentos que se pasan al método `__init__` al crear un objeto.
- `self.atributo1`, `self.atributo2`, etc.: Son los atributos del objeto que se inicializan con los valores de los argumentos.

Ejemplo del método `__init__`

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

persona1 = Persona("Juan", 30)  # Se crea un objeto Persona y se
llama al método __init__
print(persona1.nombre)  # Imprime "Juan"
print(persona1.edad)  # Imprime 30

persona2 = Persona("Ana", 25)  # Se crea otro objeto Persona
print(persona2.nombre)  # Imprime "Ana"
print(persona2.edad)  # Imprime 25
```

En este ejemplo, la clase `Persona` tiene un método `__init__` que recibe dos argumentos: `nombre` y `edad`. El método `__init__` inicializa los atributos `nombre` y `edad` del objeto con los valores de los argumentos.

En resumen

El método `__init__` es un componente fundamental en la programación orientada a objetos en Python. Permite inicializar los atributos de un objeto al momento de su creación, lo que garantiza que los objetos estén en un estado válido y listos para ser utilizados.

Método `__str__`

¿Qué es el método `__str__`?

El método `__str__` es un método especial en Python que se llama automáticamente cuando intentamos convertir un objeto a una cadena, por ejemplo, al usar la función `str()` o al imprimir el objeto con `print()`. Su principal función es devolver una representación en cadena "informal" u "amigable" del objeto, destinada al usuario final.

¿Por qué es importante el método `__str__`?

El método `__str__` es importante porque permite:

- **Personalizar la representación de objetos:** Define cómo se mostrará un objeto cuando se convierta a una cadena.
- **Hacer el código más legible:** Facilita la comprensión del código al mostrar información relevante sobre los objetos de forma clara y concisa.
- **Mejorar la experiencia del usuario:** Permite mostrar información útil y amigable sobre los objetos a los usuarios del programa.

Sintaxis del método `__str__`

La sintaxis del método `__str__` es la siguiente:

```
def __str__(self):
    return "Cadena que representa al objeto"
```

- `self`: Es una referencia al objeto que se está convirtiendo a cadena. Se utiliza para acceder a los atributos del objeto dentro del método `__str__`.

- `return "Cadena que representa al objeto"` : Es la cadena que se devuelve y que representa al objeto.

Ejemplo del método `__str__`

```
class Libro:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor

    def __str__(self):
        return f'"{self.titulo}" por {self.autor}'

libro1 = Libro("El Quijote", "Miguel de Cervantes")
print(libro1)  # Imprime "El Quijote" por Miguel de Cervantes
print(str(libro1))  # Imprime "El Quijote" por Miguel de Cervantes
```

En este ejemplo, la clase `Libro` tiene un método `__str__` que devuelve una cadena con el título y el autor del libro. Al imprimir el objeto `libro1`, se llama automáticamente al método `__str__` y se muestra la cadena formateada.

Diferencia entre `__str__` y `__repr__`

Es importante diferenciar entre los métodos `__str__` y `__repr__`:

- `__str__` : Se utiliza para crear una representación en cadena "informal" u "amigable" del objeto, destinada al usuario final.
- `__repr__` : Se utiliza para crear una representación en cadena "formal" del objeto, destinada a los desarrolladores. Esta representación debe ser lo más precisa posible y, si es posible, debería permitir reconstruir el objeto.

En general, se recomienda definir ambos métodos para personalizar la representación de los objetos en diferentes contextos.

En resumen

El método `__str__` es un componente fundamental en la programación orientada a objetos en Python. Permite personalizar la representación en cadena de un objeto, lo que facilita la comprensión del código y mejora la experiencia del usuario.

Método `__repr__`

¿Qué es el método `__repr__`?

El método `__repr__` es un método especial en Python que se llama automáticamente cuando intentamos obtener una representación en cadena "formal" de un objeto. Esto ocurre, por ejemplo, al usar la función `repr()` o al mostrar el objeto en el intérprete de Python. Su principal función es devolver una cadena que, idealmente, podría utilizarse para recrear el objeto.

¿Por qué es importante el método `__repr__`?

El método `__repr__` es importante porque permite:

- **Personalizar la representación formal de objetos:** Define cómo se mostrará un objeto cuando se solicita una representación formal del mismo.
- **Facilitar la depuración:** Facilita la comprensión del código al mostrar información detallada sobre los objetos, lo que resulta útil durante la depuración.
- **Permitir la reconstrucción de objetos:** Idealmente, la cadena devuelta por `__repr__` debería poder utilizarse para crear un nuevo objeto igual al original.

Sintaxis del método `__repr__`

La sintaxis del método `__repr__` es la siguiente:

```
def __repr__(self):
    return "Cadena que representa formalmente al objeto"
```

- `self`: Es una referencia al objeto del que se solicita la representación. Se utiliza para acceder a los atributos del objeto dentro del método `__repr__`.
- `return "Cadena que representa formalmente al objeto"`: Es la cadena que se devuelve y que representa formalmente al objeto.

Ejemplo del método `__repr__`

```
class Punto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return f"Punto({self.x}, {self.y})"

punto1 = Punto(2, 3)
print(repr(punto1))  # Imprime Punto(2, 3)
print(eval(repr(punto1)))  # Crea un nuevo objeto Punto igual a
punto1
```

En este ejemplo, la clase `Punto` tiene un método `__repr__` que devuelve una cadena con la forma `Punto(x, y)`. Esta cadena puede utilizarse para crear un nuevo objeto `Punto` con los mismos valores de `x` e `y`.

- **Teoría:**
  - Método constructor `__init__()`.
  - Métodos especiales para representación de objetos (`__str__`, `__repr__`).
- **Práctica:**
  - Implementar el método `__str__()` en la clase `Perro` para obtener una representación legible del objeto.
  - Crear una clase `CuentaBancaria` con métodos para depositar y retirar dinero.

Bloque 3: herencia

¿Qué es la herencia?

Imagina que tienes un molde para hacer galletas con forma de círculo. Ese molde sería tu "clase padre". Ahora, quieres hacer galletas con forma de estrella, pero no quieres crear un molde desde cero. Podrías usar el molde de círculo como base y simplemente agregarle la forma de estrella. ¡Eso es herencia!

En programación, la herencia permite que una clase (llamada "clase hija" o "subclase") herede atributos (datos) y métodos (acciones) de otra clase

(llamada "clase padre" o "superclase"). La clase hija puede agregar nuevos atributos y métodos, o incluso modificar los heredados para adaptarlos a sus necesidades específicas.

¿Por qué es útil la herencia?

La herencia ofrece muchas ventajas:

- **Reutilización de código:** Evita tener que escribir el mismo código una y otra vez. Si una clase ya tiene atributos y métodos que necesitas, puedes heredarlos en lugar de volver a definirlos.
- **Organización del código:** Facilita la creación de jerarquías de clases relacionadas, lo que hace que el código sea más fácil de entender y mantener.
- **Abstracción:** Permite trabajar con objetos de diferentes clases de manera más general, utilizando una interfaz común definida por la clase padre.
- **Extensibilidad:** Facilita la creación de nuevas clases que extiendan la funcionalidad de clases existentes sin tener que modificarlas directamente.

Ejemplo de herencia en Python

```
class Animal:  # Clase padre
    def __init__(self, nombre):
        self.nombre = nombre

    def comer(self):
        print("El animal está comiendo.")

class Perro(Animal):  # Clase hija que hereda de Animal
    def __init__(self, nombre, raza):
        super().__init__(nombre)  # Llama al constructor de la clase
padre
        self.raza = raza

    def ladrar(self):
        print("¡Guau!")

mi_perro = Perro("Max", "Pastor Alemán")
```

```
print(mi_perro.nombre)  # Accede al atributo heredado de Animal
mi_perro.comer()  # Llama al método heredado de Animal
mi_perro.ladrar()  # Llama al método propio de Perro
```

En este ejemplo, la clase `Perro` hereda de la clase `Animal`. `Perro` tiene un atributo propio `raza` y un método propio `ladrar()`, pero también hereda el atributo `nombre` y el método `comer()` de `Animal`.

Tipos de herencia

Existen diferentes tipos de herencia:

- **Herencia simple:** Una clase hereda de una sola clase padre.
- **Herencia múltiple:** Una clase hereda de múltiples clases padre.
- **Herencia multinivel:** Una clase hereda de otra clase, que a su vez hereda de otra clase, y así sucesivamente.

En resumen

La herencia es un concepto fundamental en la POO que permite crear nuevas clases basadas en clases existentes, lo que promueve la reutilización del código, la organización y la extensibilidad. Es una herramienta poderosa para modelar relaciones entre objetos y crear sistemas de software más complejos y eficientes.

- **Teoría:**
  - Concepto de herencia.
  - Clases base y clases derivadas.
  - Sobrescritura de métodos.
- **Práctica:**
  - Crear una clase `Animal` y clases derivadas como `Perro`, `Gato`, `Pajaro`.
  - Implementar métodos específicos para cada clase (por ejemplo, `ladrar()`, `maullar()`, `volar()`).

Bloque 4: encapsulación

¿Qué es la encapsulación?

Imagina que tienes una caja negra. No sabes cómo funciona por dentro, pero tiene una serie de botones y palancas que te permiten interactuar con ella. La encapsulación es similar: te permite ocultar los detalles internos de un objeto y exponer solo una interfaz (botones y palancas) para interactuar con él.

En programación, la encapsulación consiste en ocultar los atributos (datos) y la implementación (código) de un objeto y exponer solo una interfaz pública (métodos) para acceder a ellos y modificarlos.

¿Por qué es útil la encapsulación?

La encapsulación ofrece muchas ventajas:

- **Protección de datos:** Evita que se modifiquen los atributos de un objeto de manera accidental o incorrecta desde fuera del objeto.
- **Abstracción:** Permite trabajar con objetos sin necesidad de conocer los detalles de su implementación interna.
- **Modularidad:** Facilita la creación de módulos de código independientes y reutilizables.
- **Mantenibilidad:** Permite modificar la implementación interna de un objeto sin afectar el código que lo utiliza.

Ejemplo de encapsulación en Python

```
class Coche:
    def __init__(self, marca, modelo):
        self._marca = marca  # Atributo "protegido"
        self._modelo = modelo

    def get_marca(self):  # Método "getter" para acceder a la marca
        return self._marca

    def set_marca(self, marca):  # Método "setter" para modificar la
marca
        self._marca = marca

mi_coche = Coche("Toyota", "Corolla")
print(mi_coche.get_marca())  # Accedemos a la marca a través del
método getter
mi_coche.set_marca("Honda")  # Modificamos la marca a través del
```

```
método setter
print(mi_coche.get_marca())
```

En este ejemplo, los atributos `_marca` y `_modelo` están "protegidos" (convención con el guión bajo) para indicar que no deberían accederse directamente desde fuera de la clase. Se utilizan métodos "getter" (`get_marca()`) y "setter" (`set_marca()`) para acceder y modificar estos atributos de manera controlada.

Convenciones en Python

Python no tiene modificadores de acceso como `private` o `public` como otros lenguajes. La encapsulación se logra mediante convenciones:

- Atributos y métodos con un guión bajo al inicio (`_nombre`) se consideran "protegidos".
- Atributos y métodos con dos guiones bajos al inicio (`__nombre`) se consideran "privados" (aunque aún se puede acceder a ellos con "name mangling").

¿Qué es `@property` ?

Imagina que tienes un atributo en tu clase que quieres que se pueda acceder y modificar, pero quieres tener cierto control sobre cómo se hace. Por ejemplo, podrías querer validar el valor que se asigna al atributo o realizar alguna otra acción antes o después de acceder o modificar el atributo.

El decorador `@property` te permite hacer precisamente esto. Te permite definir un método para "obtener" el valor del atributo (getter), otro para "establecer" el valor (setter) y otro para "eliminar" el atributo (deleter). Estos métodos se comportan como si fueran atributos normales, pero te dan un mayor control sobre su acceso y modificación.

¿Por qué es útil `@property` ?

`@property` ofrece muchas ventajas:

- **Encapsulación:** Permite ocultar la implementación interna de un atributo y exponer solo una interfaz (métodos getter, setter y deleter) para interactuar con él.

- **Validación de datos:** Puedes validar los valores que se asignan a un atributo para asegurarte de que sean correctos.
- **Cálculos dinámicos:** Puedes definir atributos cuyo valor se calcula dinámicamente en función de otros atributos o datos.
- **Abstracción:** Permite trabajar con atributos como si fueran normales, pero con la posibilidad de realizar acciones adicionales cuando se acceden o modifican.

Ejemplo de `@property` en Python

```
class Persona:
    def __init__(self, nombre):
        self._nombre = nombre  # Atributo "protegido"

    @property
    def nombre(self):  # Getter para el atributo nombre
        return self._nombre

    @nombre.setter
    def nombre(self, nombre):  # Setter para el atributo nombre
        if not isinstance(nombre, str):
            raise TypeError("El nombre debe ser una cadena.")
        self._nombre = nombre

    @nombre.deleter
    def nombre(self):  # Deleter para el atributo nombre
        del self._nombre

persona = Persona("Juan")
print(persona.nombre)  # Accedemos al atributo nombre como si fuera
normal
persona.nombre = "Ana"  # Modificamos el atributo nombre como si
fuera normal
print(persona.nombre)
del persona.nombre  # Eliminamos el atributo nombre
```

En este ejemplo, el atributo `_nombre` está "protegido" (convención con el guión bajo). El decorador `@property` define un método `nombre()` que se utiliza como getter para acceder al atributo. El decorador `@nombre.setter`

define un método `nombre()` que se utiliza como setter para modificar el atributo, y el decorador `@nombre.deleter` define un método `nombre()` que se utiliza como deleter para eliminar el atributo.

- **Teoría:**
  - Concepto de encapsulación.
  - Atributos y métodos privados.
  - Propiedades.
- **Práctica:**
  - Crear una clase `Persona` con atributos privados para nombre y edad.
  - Implementar propiedades para acceder y modificar los atributos de forma controlada.

Bloque 5: polimorfismo

¿Qué es el polimorfismo?

Imagina que tienes un control remoto universal que puede controlar diferentes dispositivos: un televisor, un reproductor de DVD y un equipo de música. El control remoto tiene botones para encender, apagar y subir el volumen. Estos botones funcionan de manera diferente según el dispositivo que estés controlando, pero la interfaz (los botones) es la misma. ¡Eso es polimorfismo!

En programación, el polimorfismo permite que objetos de diferentes clases respondan al mismo mensaje o método de manera diferente. Es decir, un mismo método puede tener diferentes comportamientos según el objeto que lo llame.

¿Por qué es útil el polimorfismo?

El polimorfismo ofrece muchas ventajas:

- **Flexibilidad:** Permite escribir código que puede trabajar con objetos de diferentes clases sin necesidad de conocer su tipo específico.
- **Extensibilidad:** Facilita la creación de nuevas clases que pueden interactuar con código existente sin necesidad de modificarlo.

- **Abstracción:** Permite trabajar con objetos de manera más general, utilizando una interfaz común definida por una clase padre o interfaz.

Ejemplo de polimorfismo en Python

```
class Animal:  # Clase padre
    def __init__(self, nombre):
        self.nombre = nombre

    def hacer_sonido(self):
        print("El animal hace un sonido.")

class Perro(Animal):  # Clase hija que hereda de Animal
    def hacer_sonido(self):  # Sobreescribe el método hacer_sonido()
        print("¡Guau!")

class Gato(Animal):  # Clase hija que hereda de Animal
    def hacer_sonido(self):  # Sobreescribe el método hacer_sonido()
        print("¡Miau!")

def escuchar_animal(animal):  # Función que puede trabajar con
objetos de diferentes clases
    animal.hacer_sonido()

mi_perro = Perro("Max")
mi_gato = Gato("Misi")

escuchar_animal(mi_perro)  # Llama al método hacer_sonido() de Perro
escuchar_animal(mi_gato)  # Llama al método hacer_sonido() de Gato
```

En este ejemplo, las clases `Perro` y `Gato` heredan de la clase `Animal` y sobrescriben el método `hacer_sonido()`. La función `escuchar_animal()` puede trabajar con objetos de las clases `Perro` y `Gato` sin necesidad de saber su tipo específico, ya que ambas clases comparten el método `hacer_sonido()`.

Tipos de polimorfismo

Existen diferentes tipos de polimorfismo:

- **Polimorfismo de subtipos (o dinámico):** Se produce cuando un objeto de una clase hija se utiliza en lugar de un objeto de su clase padre.
- **Polimorfismo paramétrico:** Se produce cuando una función o clase puede trabajar con diferentes tipos de datos sin necesidad de especificar su tipo concreto.

En resumen

El polimorfismo es un concepto fundamental en la POO que permite que objetos de diferentes clases respondan al mismo mensaje o método de manera diferente. Esto promueve la flexibilidad, la extensibilidad y la abstracción en el código, lo que facilita la creación de software más complejo y adaptable.

- **Teoría:**
  - Concepto de polimorfismo.
- **Práctica:**
  - Crear una jerarquía de clases que representen figuras geométricas y que informen de su área

Bloque 6: módulos y paquetes

¿Qué es un módulo en Python?

Imagina un módulo como una caja de herramientas que contiene funciones, clases y variables que puedes utilizar en tus programas. Es una forma de organizar y reutilizar código.

En Python, un módulo es simplemente un archivo `.py` que contiene código Python. Este archivo puede definir funciones, clases y variables que puedes importar y utilizar en otros módulos o en tu programa principal.

¿Por qué usar módulos?

Los módulos ofrecen muchas ventajas:

- **Organización:** Permiten agrupar código relacionado en archivos separados, lo que hace que el código sea más fácil de entender y mantener.

- **Reutilización:** Puedes reutilizar código de módulos en diferentes programas sin tener que volver a escribirlo.
- **Modularidad:** Facilitan la creación de programas modulares, donde cada módulo se encarga de una tarea específica.
- **Abstracción:** Permiten ocultar los detalles de implementación de un módulo y exponer solo una interfaz (funciones, clases) para interactuar con él.

¿Cómo usar módulos?

Para usar un módulo en Python, primero debes importarlo utilizando la palabra clave `import`:

```
import nombre_del_modulo
```

Una vez importado, puedes acceder a las funciones, clases y variables del módulo utilizando el nombre del módulo seguido de un punto `.`:

```
nombre_del_modulo.nombre_de_la_funcion()
nombre_del_modulo.NombreDeLaClase()
nombre_del_modulo.nombre_de_la_variable
```

Ejemplo de módulo en Python

Imagina que tienes un archivo llamado `calculadora.py` que contiene las siguientes funciones:

```
# calculadora.py
def sumar(a, b):
    return a + b

def restar(a, b):
    return a - b
```

Puedes utilizar este módulo en otro archivo de la siguiente manera:

```
# mi_programa.py
import calculadora
```

```
resultado_suma = calculadora.sumar(5, 3)
resultado_resta = calculadora.restar(10, 4)

print(resultado_suma)  # Imprime 8
print(resultado_resta)  # Imprime 6
```

Módulos estándar de Python

Python viene con una gran cantidad de módulos estándar que ofrecen funcionalidades para diversas tareas, como:

- `math`: Funciones matemáticas.
- `random`: Generación de números aleatorios.
- `os`: Interacción con el sistema operativo.
- `datetime`: Manejo de fechas y horas.

¿Qué es un paquete en Python?

Imagina que tienes una caja grande llena de cajas más pequeñas. La caja grande sería tu "paquete" y las cajas pequeñas serían tus "módulos". Un paquete es una forma de agrupar módulos relacionados en un directorio.

En Python, un paquete es un directorio que contiene uno o más módulos (archivos `.py`) y un archivo especial llamado `__init__.py`. Este archivo `__init__.py` puede estar vacío o contener código de inicialización para el paquete.

¿Por qué usar paquetes?

Los paquetes ofrecen muchas ventajas:

- **Organización:** Permiten organizar módulos relacionados en una estructura jerárquica, lo que hace que el código sea más fácil de entender y mantener.
- **Espacios de nombres:** Evitan conflictos de nombres entre módulos con el mismo nombre que se encuentran en diferentes paquetes.
- **Distribución:** Facilitan la distribución de colecciones de módulos a otros desarrolladores.

¿Cómo crear un paquete?

Para crear un paquete en Python, debes seguir estos pasos:

4. Crea un directorio con el nombre de tu paquete.
5. Dentro de este directorio, crea un archivo llamado `__init__.py`. Este archivo puede estar vacío.
6. Crea los módulos (archivos `.py`) que formarán parte de tu paquete dentro del directorio.

Ejemplo de paquete en Python

Imagina que quieres crear un paquete llamado `mi_paquete` que contenga dos módulos: `matematicas.py` y `cadenas.py`. La estructura de tu directorio sería la siguiente:

```
mi_paquete/
├── __init__.py
├── matematicas.py
└── cadenas.py
```

Dentro de `matematicas.py`, podrías tener funciones como:

```python
# mi_paquete/matematicas.py
def sumar(a, b):
    return a + b

def restar(a, b):
    return a - b
```

Y dentro de `cadenas.py`, podrías tener funciones como:

```python
# mi_paquete/cadenas.py
def invertir(cadena):
    return cadena[::-1]

def longitud(cadena):
    return len(cadena)
```

¿Cómo usar paquetes?

Para usar un paquete, primero debes importarlo utilizando la palabra clave `import`:

```
import mi_paquete
```

Luego, puedes acceder a los módulos y funciones dentro del paquete utilizando la notación de punto:

```
resultado_suma = mi_paquete.matematicas.sumar(5, 3)
cadena_invertida = mi_paquete.cadenas.invertir("Hola")
```

También puedes importar módulos específicos dentro del paquete:

```
from mi_paquete import matematicas
```

Y luego usar las funciones directamente:

```
resultado_suma = matematicas.sumar(5, 3)
```

En resumen

Los módulos son una herramienta fundamental en Python que te permite organizar, reutilizar y modularizar tu código. Facilitan la creación de programas más complejos y eficientes al permitirte aprovechar el trabajo de otros desarrolladores y organizar tu propio código de manera lógica.

Los paquetes son una herramienta esencial en Python para organizar y distribuir código de manera eficiente. Permiten agrupar módulos relacionados, evitar conflictos de nombres y facilitar la reutilización del código.

- **Teoría:**
  - Concepto de módulo.
  - Creación de módulos.
  - Importación de módulos.
  - Paquetes.
- **Práctica:**

- Crear un módulo con funciones matemáticas básicas (suma, resta, multiplicación, división).
- Importar y utilizar este módulo en otro script.
- Crear un juego simple como el ahorcado utilizando clases y objetos.
- Desarrollar una pequeña aplicación de gestión de contactos.

Día 4: Bibliotecas externas y entornos virtuales

Bloque 1: biblioteca estándar

¿Qué es la biblioteca estándar?

Imagina que estás construyendo una casa. Necesitas herramientas como martillos, sierras y destornilladores. La biblioteca estándar de Python es como una caja de herramientas gigante que viene con Python. Contiene una gran cantidad de módulos (archivos con código) que te proporcionan funciones y herramientas para realizar diversas tareas.

¿Por qué es útil la biblioteca estándar?

La biblioteca estándar es útil porque te ahorra mucho tiempo y esfuerzo. En lugar de tener que escribir código desde cero para realizar tareas comunes, puedes utilizar las herramientas que ya están disponibles en la biblioteca.

¿Qué tipos de herramientas hay en la biblioteca estándar?

La biblioteca estándar es enorme y contiene módulos para casi cualquier cosa que puedas imaginar. Algunos ejemplos incluyen:

- **Manejo de archivos:** Puedes leer y escribir archivos, crear y eliminar directorios, etc.
- **Matemáticas:** Funciones matemáticas, trigonometría, números aleatorios, etc.
- **Cadenas de texto:** Manipulación de cadenas, búsqueda y reemplazo, expresiones regulares, etc.
- **Redes:** Comunicación a través de internet, creación de servidores y clientes, etc.
- **Fechas y horas:** Manipulación de fechas, horas y calendarios.

- **Sistema operativo:** Interacción con el sistema operativo, como obtener información sobre el sistema, ejecutar comandos, etc.

¿Cómo usar la biblioteca estándar?

Para usar un módulo de la biblioteca estándar, primero debes importarlo utilizando la palabra clave `import`:

```
import nombre_del_modulo
```

Una vez importado, puedes acceder a las funciones y clases del módulo utilizando el nombre del módulo seguido de un punto `.`:

```
nombre_del_modulo.nombre_de_la_funcion()
nombre_del_modulo.NombreDeLaClase()
```

Ejemplo de uso de la biblioteca estándar

```
import math  # Importamos el módulo math para funciones matemáticas

resultado = math.sqrt(25)  # Usamos la función sqrt() para calcular la raíz cuadrada de 25
print(resultado)  # Imprime 5.0
```

En resumen

La biblioteca estándar es una parte esencial de Python que te proporciona una gran cantidad de herramientas para realizar diversas tareas. Su uso te ahorra tiempo y esfuerzo, y te permite crear programas más complejos y eficientes.

- **Teoría:**
  - Biblioteca estándar vs bibliotecas externas
  - Repaso a la biblioteca estándar
- **Práctica:**
  - Guardar una lista de objetos en un archivo JSON.

Bloque 2: instalación de paquetes externos, `pip`

¿Qué es `pip`?

Imagina que estás construyendo una casa. Ya tienes las paredes y el techo, pero necesitas instalar las ventanas y las puertas. `pip` es como un "contratista" que te ayuda a encontrar e instalar las ventanas y puertas (paquetes) que necesitas para tu casa (proyecto de Python).

En términos más técnicos, `pip` es el **gestor de paquetes** de Python. Es una herramienta que te permite instalar y administrar paquetes de software escritos en Python. Estos paquetes son como "extensiones" que añaden funcionalidades a Python, como bibliotecas para trabajar con datos, gráficos, inteligencia artificial, etc.

¿Por qué es útil `pip`?

`pip` es útil porque te facilita la vida como desarrollador de Python. En lugar de tener que buscar y descargar manualmente los paquetes que necesitas, `pip` se encarga de todo el proceso.

¿Qué puedes hacer con `pip`?

- **Instalar paquetes:** Puedes instalar paquetes desde el **Python Package Index (PyPI)**, que es como una "tienda" gigante de paquetes Python.
- **Desinstalar paquetes:** Puedes desinstalar paquetes que ya no necesitas.
- **Actualizar paquetes:** Puedes actualizar paquetes a versiones más recientes.
- **Buscar paquetes:** Puedes buscar paquetes que te interesen por nombre, descripción, etc.
- **Gestionar dependencias:** `pip` se encarga de instalar las dependencias de los paquetes, es decir, otros paquetes que son necesarios para que el paquete funcione correctamente.

¿Cómo usar `pip`?

`pip` se utiliza desde la línea de comandos o terminal. Algunos comandos básicos son:

- **Instalar un paquete:**

```
pip install nombre_del_paquete
```

- Desinstalar un paquete:

```
pip uninstall nombre_del_paquete
```

- Actualizar un paquete:

```
pip install --upgrade nombre_del_paquete
```

- Mostrar información sobre un paquete:

```
pip show nombre_del_paquete
```

¿Cómo obtener `pip`?

`pip` viene instalado por defecto con las versiones modernas de Python (Python 3.4 y posteriores). Si tienes una versión anterior de Python, es posible que necesites instalar `pip` manualmente.

En resumen

`pip` es una herramienta esencial para cualquier desarrollador de Python. Facilita la gestión de paquetes, lo que te permite aprovechar el trabajo de otros desarrolladores y crear programas más complejos y eficientes.

- **Teoría:**
  - Instalador de paquetes: PIP
  - Instalando múltiples versiones de Python

Bloque 3: entornos virtuales

¿Qué es un entorno virtual?

Imagina que tienes un jardín donde quieres plantar diferentes tipos de flores. Algunas flores necesitan un suelo ácido, otras uno alcalino, y algunas no se llevan bien con ciertas plantas. Un entorno virtual es como crear pequeños jardines separados dentro de tu jardín principal. Cada jardín tiene su propio

tipo de suelo y plantas, lo que evita que las plantas compitan por recursos o se dañen entre sí.

En términos más técnicos, un entorno virtual es un directorio aislado que contiene su propia copia de Python y sus paquetes instalados. Esto significa que puedes instalar diferentes versiones de paquetes en diferentes entornos virtuales sin que interfieran entre sí.

¿Por qué usar entornos virtuales?

Los entornos virtuales ofrecen muchas ventajas:

- **Aislamiento de proyectos:** Permiten trabajar en diferentes proyectos con diferentes dependencias sin que haya conflictos entre ellos.
- **Control de versiones:** Puedes utilizar diferentes versiones de paquetes para diferentes proyectos, lo que te permite mantener la compatibilidad con proyectos antiguos.
- **Limpieza:** Mantienen tu instalación global de Python limpia y organizada, evitando la acumulación de paquetes innecesarios.
- **Portabilidad:** Facilitan la creación de entornos de desarrollo consistentes en diferentes máquinas.

¿Cómo crear un entorno virtual?

Python ofrece varias herramientas para crear entornos virtuales. La más común es `venv`, que viene incluida con Python 3.

Para crear un entorno virtual, abre la terminal y navega hasta el directorio donde quieres crear el entorno. Luego, ejecuta el siguiente comando:

```
python3 -m venv nombre_del_entorno
```

Esto creará un nuevo directorio con el nombre que hayas especificado. Dentro de este directorio, encontrarás una copia de Python y algunas herramientas para gestionar el entorno virtual.

¿Cómo activar un entorno virtual?

Para activar un entorno virtual, debes ejecutar un script que se encuentra dentro del directorio del entorno. La ubicación de este script varía según el

sistema operativo:

- **Linux/macOS:**

```
source nombre_del_entorno/bin/activate
```

- **Windows:**

```
nombre_del_entorno\Scripts\activate
```

Una vez activado, verás que el nombre del entorno aparece entre paréntesis en la línea de comandos.

¿Cómo usar paquetes dentro de un entorno virtual?

Una vez activado el entorno virtual, puedes instalar paquetes utilizando `pip` de la misma manera que lo harías normalmente:

```
pip install nombre_del_paquete
```

Los paquetes se instalarán dentro del entorno virtual y no afectarán a otros entornos ni a tu instalación global de Python.

En resumen

Los entornos virtuales son una herramienta esencial para cualquier desarrollador de Python. Permiten aislar proyectos, controlar versiones de paquetes, mantener la instalación global limpia y crear entornos de desarrollo consistentes.

- **Teoría:**
  - Módulo `venv`.
  - Generación de entornos virtuales, e instalación de paquetes.

Bloque 4: introducción a `Numpy`

¿Qué es `NumPy`?

Imagina que tienes una hoja de cálculo gigante donde necesitas realizar operaciones matemáticas complejas con grandes cantidades de datos.

`NumPy` es como esa hoja de cálculo, pero mucho más potente y eficiente.

En términos más técnicos, `NumPy` (Numerical Python) es una biblioteca de Python que proporciona herramientas para trabajar con **arrays multidimensionales** (como matrices) y realizar operaciones matemáticas de manera eficiente.

¿Por qué es útil `NumPy`?

`NumPy` es útil porque:

- **Almacenamiento eficiente:** Los arrays de `NumPy` almacenan datos de manera contigua en la memoria, lo que los hace más compactos y rápidos de acceder que las listas de Python.
- **Operaciones matemáticas optimizadas:** `NumPy` ofrece una gran cantidad de funciones matemáticas para realizar operaciones en arrays de manera rápida y eficiente, sin necesidad de escribir bucles.
- **Herramientas para álgebra lineal:** `NumPy` proporciona funciones para realizar operaciones de álgebra lineal, como multiplicación de matrices, cálculo de determinantes, resolución de sistemas de ecuaciones, etc.
- **Base para otras bibliotecas:** `NumPy` es la base para muchas otras bibliotecas populares en Python, como Pandas (para análisis de datos), `SciPy` (para computación científica) y `Matplotlib` (para visualización de datos).

¿Qué puedes hacer con `NumPy`?

- **Crear arrays:** Puedes crear arrays de diferentes dimensiones y tipos de datos.
- **Manipular arrays:** Puedes acceder a elementos, cambiar la forma, dividir, unir arrays, etc.
- **Realizar operaciones matemáticas:** Puedes realizar operaciones aritméticas, trigonométricas, exponenciales, logarítmicas, etc., en arrays de manera eficiente.
- **Realizar operaciones de álgebra lineal:** Puedes realizar operaciones con matrices, vectores, etc.
- **Trabajar con datos numéricos:** `NumPy` es ideal para trabajar con datos numéricos en cualquier campo, desde finanzas hasta física.

Ejemplo de uso de `NumPy`

```
import numpy as np

# Creamos un array de una dimensión
array1 = np.array([1, 2, 3, 4, 5])

# Creamos un array de dos dimensiones (matriz)
array2 = np.array([[1, 2, 3], [4, 5, 6]])

# Realizamos operaciones matemáticas
suma = np.sum(array1)
media = np.mean(array2)

print(suma)  # Imprime 15
print(media)  # Imprime 3.5
```

¿Qué es un array en `NumPy`?

Imagina que tienes una tabla gigante llena de números. Un array de `NumPy` es como esa tabla, pero mucho más potente y eficiente.

En términos más técnicos, un array de `NumPy` es una estructura de datos que almacena una colección de elementos del mismo tipo (números, cadenas, booleanos, etc.) en una o varias dimensiones.

¿Por qué son importantes los arrays de `NumPy`?

Los arrays de `NumPy` son importantes porque:

- **Son más eficientes que las listas:** Los arrays almacenan los datos de manera contigua en la memoria, lo que los hace más rápidos de acceder y manipular que las listas de Python.
- **Permiten operaciones matemáticas vectorizadas:** `NumPy` ofrece una gran cantidad de funciones matemáticas que pueden operar sobre arrays completos sin necesidad de escribir bucles, lo que hace que el código sea más conciso y eficiente.
- **Son la base de otras bibliotecas:** Muchas otras bibliotecas populares en Python, como Pandas (para análisis de datos), `SciPy` (para computación

científica) y `Matplotlib` (para visualización de datos), se basan en los arrays de `NumPy`.

¿Qué tipos de arrays existen?

`NumPy` ofrece arrays de diferentes dimensiones:

- **Arrays de una dimensión (vectores):** Son como listas de números.

```
import numpy as np

vector = np.array([1, 2, 3, 4, 5])
```

- **Arrays de dos dimensiones (matrices):** Son como tablas de números.

```
matriz = np.array([[1, 2, 3], [4, 5, 6]])
```

- **Arrays de más de dos dimensiones:** Pueden representar estructuras de datos más complejas.

¿Qué puedes hacer con los arrays de `NumPy`?

- **Crear arrays:** Puedes crear arrays de diferentes formas y tipos de datos.
- **Acceder a elementos:** Puedes acceder a elementos individuales o a secciones del array utilizando índices.
- **Modificar arrays:** Puedes cambiar el valor de elementos individuales o secciones del array.
- **Realizar operaciones matemáticas:** Puedes realizar operaciones aritméticas, trigonométricas, exponenciales, logarítmicas, etc., en arrays completos de manera eficiente.
- **Realizar operaciones de álgebra lineal:** Puedes realizar operaciones con matrices, vectores, etc.

Ejemplo de uso de arrays de NumPy

```
import numpy as np

# 1. Creación de arrays
```

```python
# Creamos un array de una dimensión con números del 0 al 9
vector = np.arange(10)
print("Vector:", vector)

# Creamos una matriz de 3x3 con números aleatorios entre 0 y 1
matriz = np.random.rand(3, 3)
print("Matriz:\n", matriz)

# Creamos un array de 2x2x2 con todos los elementos iguales a 1
array_3d = np.ones((2, 2, 2))
print("Array 3D:\n", array_3d)

# 2. Manipulación de arrays

# Accedemos al elemento de la fila 1 y columna 2 de la matriz
elemento = matriz[1, 2]
print("Elemento:", elemento)

# Modificamos el elemento de la fila 0 y columna 1 de la matriz
matriz[0, 1] = 10
print("Matriz modificada:\n", matriz)

# Obtenemos la forma (dimensiones) del array 3d
forma = array_3d.shape
print("Forma del array 3D:", forma)

# Redimensionamos la matriz a un vector de 9 elementos
vector_redimensionado = matriz.reshape(9)
print("Vector redimensionado:", vector_redimensionado)

# 3. Operaciones matemáticas

# Calculamos la suma de todos los elementos del vector
suma_vector = np.sum(vector)
print("Suma del vector:", suma_vector)

# Calculamos la media de los elementos de la matriz
media_matriz = np.mean(matriz)
print("Media de la matriz:", media_matriz)

# Calculamos el producto punto de la matriz y el vector
redimensionado
```

```
producto_punto = np.dot(matriz, vector_redimensionado)
print("Producto punto:", producto_punto)

# 4. Operaciones de álgebra lineal

# Calculamos la matriz transpuesta
matriz_transpuesta = matriz.T
print("Matriz transpuesta:\n", matriz_transpuesta)

# Calculamos la inversa de la matriz (si es cuadrada y tiene
inversa)
try:
    matriz_inversa = np.linalg.inv(matriz)
    print("Matriz inversa:\n", matriz_inversa)
except np.linalg.LinAlgError:
    print("La matriz no tiene inversa.")

# 5. Funciones universales

# Aplicamos la función seno a todos los elementos del vector
seno_vector = np.sin(vector)
print("Seno del vector:", seno_vector)

# Aplicamos la función exponencial a todos los elementos de la
matriz
exponencial_matriz = np.exp(matriz)
print("Exponencial de la matriz:\n", exponencial_matriz)
```

Este ejemplo muestra algunas de las funcionalidades más comunes de `NumPy`, como la creación y manipulación de arrays, operaciones matemáticas y de álgebra lineal, y el uso de funciones universales. `NumPy` es una herramienta muy potente que te permite trabajar con datos numéricos de manera eficiente y realizar tareas complejas de manera sencilla.

En resumen

`NumPy` es una herramienta fundamental para cualquier persona que trabaje con datos numéricos en Python. Su eficiencia, versatilidad y la gran cantidad de funciones que ofrece lo convierten en una herramienta imprescindible para el análisis de datos, la computación científica y muchas otras áreas.

- **Teoría:**
  - ¿Qué es `NumPy`?
  - Arrays de `NumPy`.
  - Creación de arrays.
  - Operaciones básicas con `arrays`.
- **Práctica:**
  - Crear un array de números aleatorios.
  - Realizar operaciones matemáticas básicas con arrays.

Bloque 5: visualización con `Matplotlib`

`Matplotlib`: El lienzo del artista para tus datos

Imagina que tienes un conjunto de datos fascinante y quieres compartir su historia con el mundo. `Matplotlib` es como el lienzo en blanco que te permite dar vida a tus datos a través de imágenes impactantes.

En esencia, `Matplotlib` es una biblioteca de Python que te proporciona las herramientas para crear una amplia gama de visualizaciones, desde gráficos estáticos y sencillos hasta animaciones interactivas y complejas. Es como tener un estudio de arte digital a tu disposición.

¿Por qué elegir `Matplotlib`?

`Matplotlib` se ha ganado su lugar en el corazón de la comunidad de Python por varias razones:

- **Versatilidad:** Te permite crear casi cualquier tipo de gráfico que puedas imaginar, desde líneas y barras hasta mapas de calor y visualizaciones en 3D.
- **Personalización:** Te ofrece un control granular sobre cada aspecto de tus gráficos, desde los colores y estilos hasta las etiquetas y leyendas.
- **Integración:** Se integra a la perfección con otras bibliotecas populares de Python como `NumPy` y `Pandas`, lo que te permite trabajar con datos de manera eficiente y visualizarlos de forma intuitiva.
- **Comunidad:** Cuenta con una comunidad activa y una documentación extensa, lo que facilita encontrar ayuda y ejemplos para cualquier tarea.

El poder de la visualización

`Matplotlib` te permite transformar datos brutos en información visualmente atractiva y fácil de entender. Las visualizaciones son cruciales para:

- **Explorar datos:** Identificar patrones ocultos, tendencias y relaciones en tus datos.
- **Comunicar resultados:** Compartir tus hallazgos de manera clara y efectiva con otras personas.
- **Tomar decisiones:** Basar tus decisiones en información visualmente clara y precisa.

Un vistazo a la galería de `Matplotlib`

`Matplotlib` ofrece una galería impresionante de tipos de gráficos, que incluyen:

- **Líneas:** Ideales para mostrar tendencias y cambios a lo largo del tiempo.
- **Barras:** Útiles para comparar valores entre diferentes categorías.
- **Dispersión:** Revelan la relación entre dos variables.
- **Histogramas:** Muestran la distribución de una variable.
- **Circulares (pie charts):** Ilustran la proporción de diferentes partes de un todo.
- **Área:** Similares a las líneas, pero con el área debajo rellena.
- **Cajas (box plots):** Resumen la distribución de un conjunto de datos.
- **Mapas de calor:** Representan datos con colores para identificar patrones.

Ejemplo de uso de `Matplotlib`: Un viaje a través de las líneas

```
import matplotlib.pyplot as plt

# Datos: coordenadas (x, y) de puntos en el plano
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 5, 3]

# Creamos un gráfico de líneas
plt.plot(x, y)

# Añadimos etiquetas a los ejes
plt.xlabel("Eje X")
```

```
plt.ylabel("Eje Y")

# Añadimos un título al gráfico
plt.title("Mi primer gráfico con Matplotlib")

# Mostramos el gráfico
plt.show()
```

Este código sencillo crea un gráfico de líneas que conecta los puntos (x, y) que hemos definido. ¡Es como dibujar un mapa de puntos en un lienzo digital!

En resumen

Matplotlib es una herramienta esencial para cualquier persona que trabaje con datos en Python. Su versatilidad, capacidad de personalización y la gran variedad de gráficos que ofrece la convierten en una biblioteca imprescindible para la visualización de datos.

- **Teoría:**
  - Introducción a `Matplotlib`.
  - Creación de gráficos simples (línea, barras, dispersión).
  - Personalización de gráficos.
- **Práctica:**
  - Crear un gráfico de línea para representar una función matemática.
  - Crear un gráfico de barras para mostrar datos categóricos.

Bloque 6: proyecto pequeño

- **Teoría:**
  - Revisión de los conceptos vistos en el día.
- **Práctica:**
  - Crear un programa que lea datos de un archivo CSV, los procese utilizando NumPy y los visualice con Matplotlib.

Día 5: PDF, XLSX, DOCX y bases de batos en Python

Bloque 1: introducción al manejo de archivos en Python

¿Por qué trabajar con archivos?

Los archivos son la forma en que los programas almacenan y recuperan datos de manera persistente. Imagina guardar la información de un usuario, las configuraciones de un programa o los resultados de un análisis. ¡Los archivos son la clave!

Tipos de archivos

Python te permite trabajar con diversos tipos de archivos:

- **Archivos de texto**: Contienen texto plano, como documentos `.txt` o código fuente `.py`.
- **Archivos binarios**: Almacenan datos en un formato que no es directamente legible por humanos, como imágenes `.jpg` o archivos comprimidos `.zip`.

Modos de apertura

Al abrir un archivo, debes especificar el modo en que lo harás:

- `'r'` (lectura): Abre el archivo para leer su contenido.
- `'w'` (escritura): Abre el archivo para escribir contenido. Si el archivo no existe, se crea. Si existe, se sobrescribe.
- `'a'` (añadido): Abre el archivo para añadir contenido al final.
- `'b'` (binario): Abre el archivo en modo binario.
- `'+'` (lectura y escritura): Abre el archivo para ambas operaciones.

Apertura y cierre de archivos

La forma más común de abrir un archivo es con la función `open()`:

```
archivo = open("nombre_del_archivo.txt", "r")  # Abre el archivo en modo lectura
```

Es crucial cerrar el archivo una vez que hayas terminado de usarlo para liberar los recursos del sistema:

```
archivo.close()
```

Lectura de archivos

Puedes leer el contenido de un archivo de varias maneras:

- `read()`: Lee todo el contenido del archivo y lo devuelve como una cadena.
- `readline()`: Lee una línea del archivo y la devuelve como una cadena.
- `readlines()`: Lee todas las líneas del archivo y las devuelve como una lista de cadenas.

```
contenido = archivo.read()  # Lee todo el contenido
linea = archivo.readline()  # Lee una línea
lineas = archivo.readlines()  # Lee todas las líneas
```

Escritura de archivos

Para escribir en un archivo, ábrelo en modo escritura (`'w'`) o añadido (`'a'`) y utiliza el método `write()`:

```
archivo.write("Este es el contenido que quiero escribir.\n")
```

El manejo de excepciones

Al trabajar con archivos, pueden surgir errores (como que el archivo no exista). Es importante manejar estos errores para evitar que tu programa falle:

```
try:
    archivo = open("archivo_inexistente.txt", "r")
    contenido = archivo.read()
    archivo.close()
except FileNotFoundError:
    print("¡El archivo no existe!")
```

El uso de `with`

La forma más segura y elegante de trabajar con archivos es utilizando la declaración `with`. Esto asegura que el archivo se cierre automáticamente, incluso si ocurre un error:

```
with open("nombre_del_archivo.txt", "r") as archivo:
    contenido = archivo.read()
    # No es necesario cerrar el archivo, se cierra automáticamente
al salir del bloque `with`
```

Ejemplos prácticos

- **Lectura de un archivo línea por línea**:

```
with open("archivo.txt", "r") as archivo:
    for linea in archivo:
        print(linea)
```

- **Escritura de datos en un archivo**:

```
datos = ["Juan", "Ana", "Carlos"]
with open("nombres.txt", "w") as archivo:
    for nombre in datos:
        archivo.write(nombre + "\n")
```

¿Qué es el módulo `pathlib`?

Imagina que cada archivo y directorio en tu computadora es como una pieza de Lego. El módulo `pathlib` te proporciona herramientas para construir y manipular "caminos" (rutas) entre estas piezas de Lego de manera intuitiva y multiplataforma.

En términos más técnicos, `pathlib` es un módulo que define la clase `Path`, que representa una ruta a un archivo o directorio. Esta clase ofrece métodos y propiedades para realizar diversas operaciones con rutas de manera más legible y orientada a objetos que el módulo `os.path`.

¿Por qué usar el módulo `pathlib`?

El módulo `pathlib` te facilita la vida como programador porque:

- **Hace que tu código sea más legible:** La sintaxis orientada a objetos de `pathlib` hace que el código sea más fácil de entender y mantener.

- **Aumenta la portabilidad:** Al igual que `os.path`, `pathlib` te ayuda a escribir código que funcione correctamente en diferentes sistemas operativos.
- **Ofrece una interfaz más intuitiva:** Los métodos y propiedades de la clase `Path` son más descriptivos y fáciles de recordar que las funciones de `os.path`.

Funciones clave del módulo `pathlib`

Aquí tienes algunas de las funcionalidades más utilizadas del módulo `pathlib`:

- **Creación de objetos `Path`:** Puedes crear objetos `Path` que representan rutas a archivos o directorios.

```
from pathlib import Path

ruta = Path("/home/usuario/documentos/informe.txt")
```

- **Unión de rutas:** Puedes unir componentes de una ruta utilizando el operador `/`.

```
ruta = Path("/home/usuario") / "documentos" / "informe.txt"
```

- **Obtención de información sobre rutas:** Puedes obtener información sobre rutas utilizando propiedades como `name`, `stem`, `suffix`, `parent`, etc.

```
nombre_archivo = ruta.name  # informe.txt
nombre_sin_extension = ruta.stem  # informe
extension = ruta.suffix  # .txt
directorio_padre = ruta.parent  # /home/usuario/documentos
```

- **Comprobación de existencia:** Puedes comprobar si un archivo o directorio existe utilizando el método `exists()`.

```
es_archivo = ruta.is_file()
```

```
es_directorio = ruta.is_dir()
```

- **Otras operaciones:** `pathlib` ofrece muchos otros métodos para realizar diversas operaciones con rutas, como crear directorios, copiar archivos, cambiar nombres, etc.

Ejemplo práctico

Imagina que quieres crear un programa que busque todos los archivos `.txt` en un directorio y los copie a otro directorio. Podrías usar el módulo `pathlib` para realizar esta tarea de manera eficiente:

```
from pathlib import Path
import shutil

def copiar_archivos_txt(directorio_origen, directorio_destino):
    for archivo in directorio_origen.iterdir():
        if archivo.is_file() and archivo.suffix == ".txt":
            shutil.copy(archivo, directorio_destino)

directorio_origen = Path("/home/usuario/documentos")
directorio_destino = Path("/home/usuario/copias")
copiar_archivos_txt(directorio_origen, directorio_destino)
```

- **Teoría:**
  - Modos de apertura de archivos (lectura, escritura, append).
  - Funciones básicas para trabajar con archivos (open, read, write, close).
  - Conceptos de rutas y directorios.
- **Práctica:**
  - Crear un archivo de texto y escribir en él.
  - Leer un archivo de texto y mostrar su contenido por pantalla.
  - Copiar un archivo a otra ubicación.

Bloque 2: interacción con archivos PDF

¿Qué es `pypdf`?

`pypdf` es una biblioteca de Python de código abierto que te permite manipular archivos PDF. Con `pypdf`

¿Por qué usar `pypdf`?

`pypdf` es útil porque:

- Es fácil de usar y tiene una sintaxis clara.
- Es multiplataforma, lo que significa que funciona en Windows, macOS y Linux.
- Es de código abierto y gratuito.
- Es una versión actualizada y mejorada de PyPDF2.

¿Cómo instalar `pypdf`?

```
pip install pypdf
```

Puedes realizar una variedad de tareas, como:

1. Lectura de PDFs:

- **Extracción de texto:**
  - Puedes extraer texto de páginas individuales o de todo el documento.
  - Esto es útil para analizar el contenido textual de PDFs, buscar palabras clave o realizar procesamiento de lenguaje natural.

```
from pypdf import PdfReader

def extraer_texto_pdf(nombre_archivo):
    """Extrae el texto de todas las páginas de un archivo PDF."""
    with open(nombre_archivo, 'rb') as archivo_pdf:
        lector_pdf = PdfReader(archivo_pdf)
        texto_completo = ""
        for pagina in lector_pdf.pages:
            texto_completo += pagina.extract_text() or '' # Añadido
'or '' para manejar páginas vacías
        return texto_completo

# Uso de la función
```

```
texto_extraido = extraer_texto_pdf("mi_archivo.pdf")
print(texto_extraido)
```

- **Obtención de metadatos:**
  - Accede a información como el autor, título, creador, productor y fecha de creación del PDF.
  - Esto te permite obtener detalles sobre el documento y su origen.

```
from pypdf import PdfReader

def obtener_metadatos_basicos(nombre_archivo):
    """Obtiene metadatos básicos de un archivo PDF."""
    with open(nombre_archivo, 'rb') as archivo_pdf:
        lector_pdf = PdfReader(archivo_pdf)
        metadatos = lector_pdf.metadata
        if metadatos:
            return metadatos
        else:
            return "No se encontraron metadatos en el PDF."

# Uso de la función
metadatos = obtener_metadatos_basicos("mi_archivo.pdf")
print(metadatos)
```

- **División de PDFs:**
  - Separa un PDF en páginas individuales o en rangos de páginas.
  - Esto es útil para extraer secciones específicas de un documento grande.

```
from pypdf import PdfReader, PdfWriter

def dividir_pdf(nombre_archivo_entrada, directorio_salida):
    """Divide un archivo PDF en páginas individuales."""
    try:
        with open(nombre_archivo_entrada, 'rb') as archivo_entrada:
            lector_pdf = PdfReader(archivo_entrada)
            for numero_pagina in range(len(lector_pdf.pages)):
                escritor_pdf = PdfWriter()
```

```
escritor_pdf.add_page(lector_pdf.pages[numero_pagina])
                nombre_archivo_salida = f"
{directorio_salida}/pagina_{numero_pagina + 1}.pdf"
                with open(nombre_archivo_salida, 'wb') as
archivo_salida:
                    escritor_pdf.write(archivo_salida)
        print(f"El archivo '{nombre_archivo_entrada}' se ha dividido
en páginas individuales en el directorio '{directorio_salida}'.")
    except FileNotFoundError:
        print(f"Error: El archivo '{nombre_archivo_entrada}' no se
encontró.")
    except Exception as e:
        print(f"Ocurrió un error inesperado: {e}")

# Uso de la función
nombre_archivo_entrada = "mi_archivo.pdf"  # Reemplaza con la ruta
de tu archivo PDF
directorio_salida = "paginas_pdf"  # Reemplaza con el directorio de
salida deseado

# Crear el directorio de salida si no existe
import os
os.makedirs(directorio_salida, exist_ok=True)

dividir_pdf(nombre_archivo_entrada, directorio_salida)
```

2. Escritura de PDFs:

- **Creación de PDFs desde cero:**
  - Genera nuevos archivos PDF y añade contenido como texto e imágenes.
  - Esto te permite crear informes, facturas u otros documentos PDF personalizados.
- **Añadir páginas:**
  - Combina páginas de diferentes PDFs en un solo documento.
  - Esto es útil para fusionar varios documentos en un solo archivo.
- **Rotación de páginas:**
  - Cambia la orientación de las páginas.

    - Esto permite ajustar la orientación de las páginas dentro del documento.
- **Añadir marcas de agua:**
    - Superpón texto o imágenes a las páginas.
    - Útil para proteger documentos o añadir identificación.
- **Actualizar metadatos PDF:**
    - También puedes actualizar los metadatos de los documentos PDF usando pypdf.

3. Modificación de PDFs existentes:

- **Cifrado y descifrado de PDFs:**
    - Protege y desbloquea PDFs con contraseñas.
    - Esto te permite controlar el acceso a documentos confidenciales.
- **Fusión de PDFs:**
    - Combina varios PDFs en un solo documento.
    - Similar a añadir páginas, pero con opciones adicionales para el orden y la estructura.
- **Recorte de páginas:**
    - Elimina partes no deseadas de las páginas.
    - Esto permite ajustar el área visible de las páginas.
- **Añadir anotaciones:**
    - Agregar anotaciones de formas: podemos dibujar formas como líneas, rectángulos, elipses y polígonos, etc.
    - Agregar anotaciones de texto: Podemos agregar anotaciones de texto en posiciones específicas de páginas PDF. 1

4. Otras funcionalidades:

- **Extracción de imágenes:**
    - Obtén las imágenes incrustadas en un PDF.
    - Esto te permite recuperar elementos visuales de los documentos.
- **Manipulación de marcadores, anotaciones y campos de formulario.**
- **Teoría:**
    - Introducción a la librería PyPDF2.
    - Extracción de texto de un PDF.

    - Búsqueda de texto dentro de un PDF.
- **Práctica:**
    - Extraer todas las páginas de un PDF y guardarlas como imágenes.
    - Buscar una palabra clave específica en un PDF y mostrar la página donde se encuentra.

Bloque 3: interacción con archivos XLSX

¿Qué es `openpyxl`?

`openpyxl` es una biblioteca de Python que te permite:

- **Leer archivos Excel**: Extraer datos de hojas de cálculo existentes.
- **Escribir archivos Excel**: Crear nuevas hojas de cálculo o modificar las existentes.
- **Manipular hojas de cálculo**: Añadir, eliminar, renombrar y mover hojas.
- **Formatear celdas**: Cambiar el estilo, color, fuente y otros atributos de las celdas.
- **Añadir gráficos**: Crear y añadir gráficos a las hojas de cálculo.

¿Por qué usar `openpyxl`?

- **Automatización**: Permite automatizar tareas repetitivas en Excel, como la generación de informes o la manipulación de datos.
- **Análisis de datos**: Facilita la extracción y manipulación de datos de hojas de cálculo para su análisis.
- **Generación de informes**: Permite generar informes personalizados en formato Excel.

¿Cómo instalar `openpyxl`?

Puedes instalar `openpyxl` utilizando `pip`:

```
pip install openpyxl
```

Ejemplos de uso:

1. Leer un archivo Excel:

```
import openpyxl

def leer_excel(nombre_archivo):
    """Lee un archivo Excel y devuelve los datos como una lista de
listas."""
    libro = openpyxl.load_workbook(nombre_archivo)
    hoja = libro.active
    datos = []
    for fila in hoja.iter_rows(values_only=True):
        datos.append(fila)
    return datos

datos_excel = leer_excel("mi_archivo.xlsx")
print(datos_excel)
```

2. Escribir un archivo Excel:

```
import openpyxl

def escribir_excel(nombre_archivo, datos):
    """Escribe datos en un archivo Excel."""
    libro = openpyxl.Workbook()
    hoja = libro.active
    for fila_idx, fila in enumerate(datos, start=1):
        for col_idx, valor in enumerate(fila, start=1):
            hoja.cell(row=fila_idx, column=col_idx, value=valor)
    libro.save(nombre_archivo)

datos_para_escribir = [["Nombre", "Edad"], ["Juan", 30], ["Ana",
25]]
escribir_excel("mi_archivo_escrito.xlsx", datos_para_escribir)
```

3. Manipular celdas:

```
import openpyxl

def manipular_celdas(nombre_archivo):
    """Manipula celdas en un archivo Excel."""
    libro = openpyxl.load_workbook(nombre_archivo)
    hoja = libro.active
```

```
    hoja["A1"] = "Nuevo valor"
    hoja.cell(row=2, column=2, value="Valor modificado")
    libro.save(nombre_archivo)

manipular_celdas("mi_archivo_escrito.xlsx")
```

- **Teoría:**
  - Introducción a la librería openpyxl.
  - Leer datos de un archivo Excel.
  - Escribir datos en un archivo Excel.
  - Manipulación de hojas de cálculo.
- **Práctica:**
  - Leer un archivo Excel y convertir los datos en un DataFrame de Pandas.
  - Crear un nuevo archivo Excel y escribir datos en él.

Bloque 4: interacción con archivos DOCX

¿Qué es `python-docx`?

`python-docx` es una biblioteca de Python que te permite:

- **Crear documentos .docx**: Generar documentos de Word desde cero.
- **Modificar documentos .docx**: Abrir documentos existentes, editar su contenido y guardar los cambios.
- **Extraer información de documentos .docx**: Leer el contenido de documentos de Word y extraer texto, imágenes y otros elementos.

¿Por qué usar `python-docx`?

- **Automatización**: Permite automatizar la creación y modificación de documentos de Word, lo que ahorra tiempo y esfuerzo.
- **Generación de informes**: Facilita la generación de informes personalizados en formato Word.
- **Manipulación de contenido**: Permite manipular el contenido de documentos de Word de manera programática.

¿Cómo instalar `python-docx`?

Puedes instalar `python-docx` utilizando `pip`:

```
pip install python-docx
```

Ejemplos de uso:

1. Crear un documento .docx:

```
from docx import Document

documento = Document()

documento.add_heading('Mi primer documento de Word', 0)

p = documento.add_paragraph('Un párrafo simple con algo de texto ')
p.add_run('en negrita.').bold = True
p.add_run(' y algo de texto ')
p.add_run('en cursiva.').italic = True

documento.add_page_break()

documento.save('mi_documento.docx')
```

2. Modificar un documento .docx existente:

```
from docx import Document

documento = Document('mi_documento.docx')

p = documento.add_paragraph('Este es un párrafo añadido al final del documento.')

documento.save('mi_documento_modificado.docx')
```

3. Extraer texto de un documento .docx:

```
from docx import Document

documento = Document('mi_documento.docx')
```

```
for párrafo in documento.paragraphs:
    print(párrafo.text)
```

- **Teoría:**
  - Introducción a la librería python-docx.
  - Lectura y escritura de documentos Word.
  - Manipulación de estilos, tablas e imágenes.
- **Práctica:**
  - Crear un documento Word simple y agregar texto, tablas e imágenes.
  - Extraer el texto de un documento Word existente.

Bloque 5: introducción a SQLite

SQLite es un sistema de gestión de bases de datos relacional (RDBMS) contenido en una pequeña biblioteca escrita en C. A diferencia de otros sistemas de gestión de bases de datos, SQLite no tiene un proceso de servidor separado. Lee y escribe directamente en archivos de disco ordinarios. Una base de datos SQLite completa se almacena en un único archivo de disco.

Características clave de SQLite:

- **Ligero:**
  - SQLite es una biblioteca pequeña y eficiente, lo que la hace ideal para dispositivos con recursos limitados, como teléfonos móviles y sistemas integrados.
- **Sin servidor:**
  - SQLite no requiere un proceso de servidor separado. Esto simplifica la configuración y el mantenimiento.
- **Autocontenido:**
  - Una base de datos SQLite completa se almacena en un solo archivo de disco, lo que facilita su copia y distribución.
- **Multiplataforma:**
  - SQLite funciona en una amplia variedad de sistemas operativos, incluidos Windows, macOS, Linux y Android.
- **Compatible con ACID:**

    - SQLite cumple con las propiedades ACID (atomicidad, consistencia, aislamiento y durabilidad), lo que garantiza la integridad de los datos.
- **Lenguaje SQL:**
    - SQLite utiliza SQL (Structured Query Language) para gestionar y consultar datos.

Usos comunes de SQLite:

- **Aplicaciones móviles:**
    - SQLite es ampliamente utilizado en aplicaciones móviles para almacenar datos localmente en dispositivos.
- **Aplicaciones de escritorio:**
    - SQLite se puede utilizar en aplicaciones de escritorio para almacenar datos de configuración, datos de usuario y otros datos locales.
- **Sistemas integrados:**
    - SQLite es ideal para sistemas integrados debido a su tamaño reducido y su bajo consumo de recursos.
- **Desarrollo web:**
    - SQLite se puede utilizar para sitios web pequeños o para prototipos de aplicaciones web.
- **Almacenamiento local de datos:**
    - SQLite es muy útil para almacenar datos localmente, para aplicaciones que no requieren de un servidor de base de datos.

Ventajas de SQLite:

- **Facilidad de uso:**
    - SQLite es fácil de configurar y utilizar.
- **Rendimiento:**
    - SQLite puede ser muy rápido para ciertas cargas de trabajo.
- **Portabilidad:**
    - SQLite se puede utilizar en casi cualquier plataforma.

Desventajas de SQLite:

- **Concurrencia:**
  - SQLite no es ideal para aplicaciones con alta concurrencia.
- **Escalabilidad:**
  - SQLite no es adecuado para grandes conjuntos de datos o aplicaciones con muchos usuarios.

En resumen, SQLite es una base de datos ligera, sin servidor y fácil de usar que es ideal para una amplia variedad de aplicaciones.

Aquí tienes algunos ejemplos de cómo conectar y trabajar con SQLite en Python usando el módulo `sqlite3`:

Ejemplo 1: Conexión y creación de una tabla

```
import sqlite3

def crear_tabla(nombre_base_datos):
    """Crea una tabla en una base de datos SQLite."""
    try:
        conexion = sqlite3.connect(nombre_base_datos)
        cursor = conexion.cursor()
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS empleados (
                id INTEGER PRIMARY KEY,
                nombre TEXT NOT NULL,
                departamento TEXT,
                salario REAL
            )
        ''')
        conexion.commit()
        print("Tabla 'empleados' creada con éxito.")
    except sqlite3.Error as error:
        print("Error al crear la tabla:", error)
    finally:
        if conexion:
            conexion.close()

# Uso de la función
crear_tabla("mi_base_de_datos.db")
```

Ejemplo 2: Inserción de datos

```
import sqlite3

def insertar_datos(nombre_base_datos, datos):
    """Inserta datos en la tabla 'empleados'."""
    try:
        conexion = sqlite3.connect(nombre_base_datos)
        cursor = conexion.cursor()
        cursor.execute('''
            INSERT INTO empleados (nombre, departamento, salario)
            VALUES (?, ?, ?)
        ''', datos)
        conexion.commit()
        print("Datos insertados con éxito.")
    except sqlite3.Error as error:
        print("Error al insertar datos:", error)
    finally:
        if conexion:
            conexion.close()

# Uso de la función
datos_empleado = ("Ana García", "Ventas", 35000.00)
insertar_datos("mi_base_de_datos.db", datos_empleado)
```

Ejemplo 3: Consulta de datos

```
import sqlite3

def consultar_datos(nombre_base_datos, departamento):
    """Consulta empleados de un departamento específico."""
    try:
        conexion = sqlite3.connect(nombre_base_datos)
        cursor = conexion.cursor()
        cursor.execute('''
            SELECT * FROM empleados WHERE departamento = ?
        ''', (departamento,))
        resultados = cursor.fetchall()
        if resultados:
            for fila in resultados:
                print(fila)
```

```
        else:
            print(f"No se encontraron empleados en el departamento
'{departamento}'.")
    except sqlite3.Error as error:
        print("Error al consultar datos:", error)
    finally:
        if conexion:
            conexion.close()

# Uso de la función
consultar_datos("mi_base_de_datos.db", "Ventas")
```

Explicación y consideraciones importantes:

- **Importar `sqlite3`:** Importamos el módulo `sqlite3` para trabajar con SQLite.
- **Conexión a la base de datos:**
  - `sqlite3.connect(nombre_base_datos)` establece una conexión con la base de datos. Si la base de datos no existe, se crea un nuevo archivo.
- **Creación de un cursor:**
  - `conexion.cursor()` crea un objeto cursor que permite ejecutar comandos SQL.
- **Ejecución de comandos SQL:**
  - `cursor.execute(comando_sql, parametros)` ejecuta un comando SQL con parámetros opcionales.
  - Usamos `?` como marcador de posición para los parámetros y pasamos los valores como una tupla.
- **Confirmación de cambios:**
  - `conexion.commit()` guarda los cambios realizados en la base de datos.
- **Cierre de la conexión:**
  - `conexion.close()` cierra la conexión con la base de datos.
- **Manejo de errores:**
  - Usamos un bloque `try...except...finally` para capturar posibles errores y cerrar la conexión en cualquier caso.
- **Teoría:**

    - ¿Qué es SQLite?
    - Conexión a una base de datos SQLite.
    - Creación de tablas.
    - Inserción, actualización y eliminación de datos.
    - Consultas SQL básicas.
- **Práctica:**
    - Crear una base de datos SQLite para almacenar información de contactos.
    - Insertar, actualizar y eliminar registros.
    - Realizar consultas para obtener información específica.

Bloque 6: integración de archivos y bases de datos

- **Teoría:**
    - Importar datos de un archivo (CSV, Excel) a una base de datos.
    - Exportar datos de una base de datos a un archivo.
- **Práctica:**
    - Importar datos de un archivo CSV a una tabla de SQLite.
    - Exportar los datos de una tabla de SQLite a un archivo Excel.